

Specification and Verification of Real-Time Systems Using the OTS/CafeOBJ Method: A Tutorial

Masaki Nakamura

Department of Data Science, Faculty of Information Engineering
Toyama Prefectural University, Japan
`masaki-n@pu-toyama.ac.jp`

SICE FES 2026 Tutorial

Outline

1. Introduction
2. The OTS/CafeOBJ Method Overview
3. Modeling Multitask Real-Time Systems
in OTS/CafeOBJ Specifications
4. Case study: Fischer's protocol
5. Case study: autonomous vehicle control system
6. Conclusion

1 Introduction

- **Formal methods** are mathematically based techniques for specifying and verifying software and hardware systems.
- **Formal specification languages** are languages for describing systems and desired properties precisely.
- **CafeOBJ** is an executable algebraic specification language developed in the 1990s in the tradition of Clear, OBJ2, and OBJ3, supporting specification execution by term rewriting [1, 2].
- **2000s:** The OTS/CafeOBJ method combines observational transition systems (OTSs), CafeOBJ, and proof scores for system verification [2–4].

1.1 The OTS/CafeOBJ method

The OTS/CafeOBJ method proceeds through the following steps.

- ❶ **Model the system as an OTS**
- ❷ **Describe its specification in CafeOBJ**
- ❸ **Verify desired properties by proof scores**

Verification in the OTS/CafeOBJ method is based on theorem proving. Unlike exhaustive state-space exploration such as model checking, proof scores can reason about abstract models, including multitask systems with an arbitrary number of processes.

1.2 Development toward real-time and hybrid systems

This tutorial focuses on the development of the OTS/CafeOBJ method for real-time and hybrid systems.

- **OTS/CafeOBJ method (2003)** [3]

Early case studies mainly concerned discrete systems, including multitask systems.

- **Multitask real-time OTS/CafeOBJ method (2007, 2022)** [5, 6]

The method was extended to systems with real-time constraints, including lower and upper time bounds on system behavior.

- **Multitask hybrid OTS/CafeOBJ method (2026)** [7]

The method was further extended to hybrid systems combining discrete behavior and continuous dynamics.

1.3 Getting started with CafeOBJ

CafeOBJ can be used interactively from the command line to select a module, inspect it, and reduce a term.

```
$ cafeobj
...
CafeOBJ> select BOOL .
BOOL> show
sys:mod! BOOL
...
BOOL> red X:Bool or not X:Bool .
(true):Bool
```

- `select BOOL .` selects the built-in module `BOOL` as the current module.
- `show` displays the declarations of the current module.
- `red` reduces a term using the equations available in the current module.

`BOOL` provides sort `Bool`, constants `true` and `false`, and Boolean operations such as `not`, `and`, and `or`.

Available for Windows, macOS, and Linux.

<https://cafeobj.org/download/>

Outline

1. Introduction
- 2. The OTS/CafeOBJ Method Overview**
3. Modeling Multitask Real-Time Systems
in OTS/CafeOBJ Specifications
4. Case study: Fischer's protocol
5. Case study: autonomous vehicle control system
6. Conclusion

2 The OTS/CafeOBJ Method Overview

This section introduces the notions and notations used to specify and verify multitask systems as OTSs in CafeOBJ.

2.1 CafeOBJ specifications for abstract data types

2.2 CafeOBJ specifications for multitask systems

2.3 Verification with proof scores

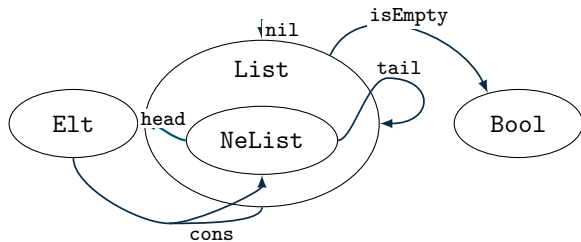
Note on the Proof Scores. The organization of some Proof Scores used in these slides differs slightly from that in the tutorial paper. The slides consistently use the accompanying executable Proof Scores unless otherwise stated.

The following slides are based mainly on [1]–[4].

2.1 Abstract Data Types as Order-Sorted Algebras

Order-sorted algebras model abstract data types specified by signatures and equations.

- An abstract data type describes kinds of data and their operations independently of implementation.
- Algebraic specifications represent kinds of data as sorts and operations on those data.
- In a model (order-sorted algebra), sorts are interpreted as carrier sets and operations as functions between them.

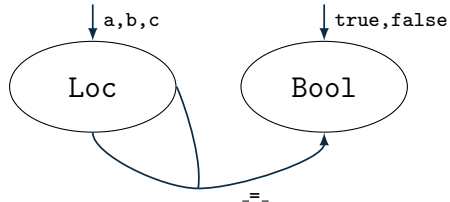


Abstract data type for lists

2.1.1 CafeOBJ specifications for abstract data types

Suppose we want to specify a sort `Loc` with three values `a`, `b`, and `c`, together with an operation `_ = _` that takes two `Loc` values and returns a Boolean value. CafeOBJ describes such a data type by a module.

Data type to be specified



`a, b, c : -> Loc`

`_ = _ : Loc Loc -> Bool`

We use the built-in module `BOOL` for Boolean values.

Example 1: CafeOBJ specification of `Loc`

```
mod! LOC {  
  [ Loc ]  
  ops a b c : -> Loc  
  op _=_ : Loc Loc -> Bool {comm}  
  var L : Loc  
  eq (L = L) = true .  
  eq (a = b) = false .  
  eq (a = c) = false .  
  eq (b = c) = false .  
}
```

LOC.cafe

Note. The built-in module `BOOL` is implicitly imported by all CafeOBJ modules.

2.1.2 Structure of a CafeOBJ module

A CafeOBJ module consists of a signature Σ and a set of axioms E .

```
mod! LOC { LOC.cafe
  [ Loc ]
  ops a b c : -> Loc
  op _=_ : Loc Loc -> Bool {comm}

  var L : Loc
  eq (L = L) = true .
  eq (a = b) = false .
  eq (a = c) = false .
  eq (b = c) = false .
}
```

Σ_{LOC}

The signature, consisting of sort and operation declarations.

E_{LOC}

The axioms part, consisting of variable declarations and equations.

2.1.3 Signature of LOC

A signature consists of a set of sorts, an order on the sorts, and a set of operation symbols.

```
mod! LOC {  
  [ Loc ]  
  ops a b c : -> Loc  
  op _=_ : Loc Loc -> Bool {comm}  
  var L : Loc  
  eq (L = L) = true .  
  eq (a = b) = false .  
  eq (a = c) = false .  
  eq (b = c) = false .  
}
```

LOC.cafe

- $S_{\text{LOC}} = \{\text{Loc}\}$
- $\leq_{\text{LOC}}$: no proper subsort relation.
- F_{LOC} contains:
 - constants a, b, c ;
 - operation $_=_$.
- $\{\text{comm}\}$ is an operation attribute declaring $_=_$ commutative.
- Thus, $X = Y$ and $Y = X$ are treated equivalently.

2.1.4 Terms over a Signature

A signature determines how terms are recursively constructed.

Recursive construction

Let X be an S -sorted set of variables.

- 1 If $x \in X_s$, then x is a term of sort s .
- 2 If $c : \rightarrow s$ is a constant, then c is a term of sort s .
- 3 If $\sigma : s_1 \cdots s_n \rightarrow s$ and each t_i is a term of sort s_i , then $\sigma(t_1, \dots, t_n)$ is a term of sort s .
- 4 If $s_0 \leq s$, every term of the subsort s_0 is also a term of its supersort s .

LOC signature fragment

```
ops a b c : -> Loc      LOC.cafe (excerpt)
op _=_ : Loc Loc -> Bool {comm}
```

Ground terms (no variables)

```
a, b, c : Loc
true, false : Bool
(a = b), ... : Bool
```

Based on [2], Sec. 3.

2.1.5 Equations of LOC

Equations specify equalities between terms of the same sort.

```
mod! LOC {  
  [ Loc ]  
  ops a b c : -> Loc  
  op _=_ : Loc Loc -> Bool {comm}  
  var L : Loc  
  eq (L = L) = true .  
  eq (a = b) = false .  
  eq (a = c) = false .  
  eq (b = c) = false .  
}
```

LOC.cafe

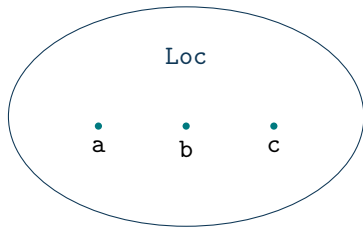
$\underbrace{(L = L)}_{\text{term of sort Bool}} = \underbrace{\text{true}}_{\text{term of sort Bool}}$

- L denotes an arbitrary term t of sort Loc .
- Thus, `eq (L = L) = true` .
declares that, for any term t of sort Loc , $(t = t) = \text{true}$.
- The remaining equations distinguish the constants `a`, `b`, and `c`.

2.1.6 Semantics of LOC: the Initial Model

The keyword `mod!` declares a tight module with initial semantics; it denotes the initial model (initial algebra) of the specification.

- **No junk:** every element is denoted by a ground term.
- **No confusion:** distinct ground terms denote distinct elements unless the axioms identify them.



In the initial model of `LOC`, `a`, `b`, and `c` denote exactly three distinct elements of `Loc`.

2.1.7 Specification Execution by Rewriting

CafeOBJ executes a specification by using its equations as left-to-right rewrite rules, together with declared operation attributes, and reducing terms to normal forms.

```
open LOC .  
  red b = b .  
  red c = a .  
close
```

2.1.7.cafe (excerpt)

CafeOBJ> in 2.1.7.cafe

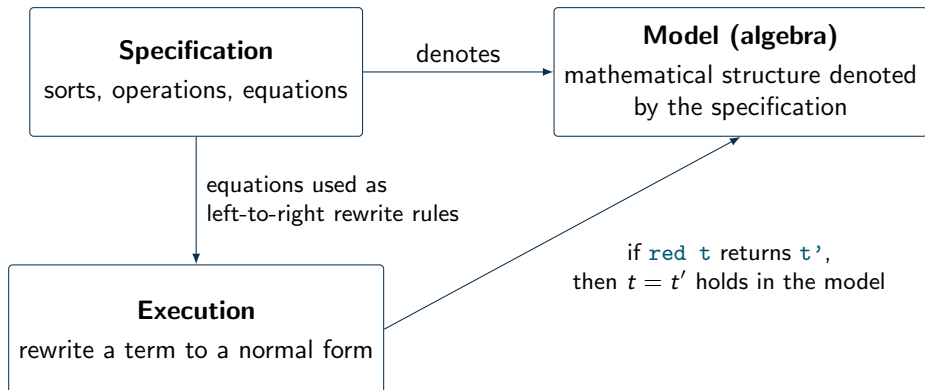
```
...  
(true):Bool  
(false):Bool
```

- **open LOC**: open **LOC** as a temporary working module; additional operations and equations can be declared there.
- **red t**: reduce term *t* to a normal form by applying equations as left-to-right rewrite rules.

- **b = b** matches
eq (L = L) = true .
result: true.
- By {comm}, **c = a** matches
eq (a = c) = false .
result: false.

Specification, Model, and Execution

The result of CafeOBJ execution is guaranteed to be valid in the model denoted by the specification.



2.1.8 Example 2: an arbitrary process-ID type

The module `PID` specifies an arbitrary set with a binary predicate.

```
mod* PID {  
  [ Pid ]  
  op _=_ : Pid Pid -> Bool {comm}  
}
```

PID.cafe

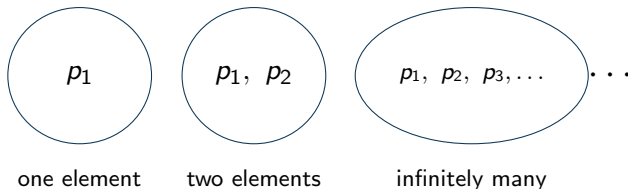
- `Pid`: the sort of process identifiers.
- `_=_`: a binary predicate on `Pid`.
- No operations other than the equality predicate are declared.
- No equations are declared.

2.1.9 Semantics of PID: Models of a Loose Specification

The keyword `mod*` declares a loose module that denotes all models satisfying the declaration.

```
PID.cafe
mod* PID {
  [ Pid ]
  op _=_ : Pid Pid -> Bool {comm}
}
```

Examples of possible `Pid` carriers



`PID` denotes many possible models, including models with different numbers of process identifiers.

2.2 CafeOBJ specifications for multitask systems

Section 2.1 described CafeOBJ specifications of abstract data types.

We now consider specifications of systems that change state through transitions.

We first model such a system as an observational transition system (OTS).

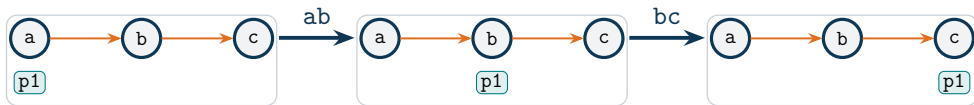
An OTS is described in CafeOBJ as a behavioral specification.

OTS/CafeOBJ specifications are a subclass of behavioral specifications.

2.2.1 The ABC System

In this tutorial, we use the following ABC system as a running example.

A process moves through three locations by transitions **ab** and **bc**.

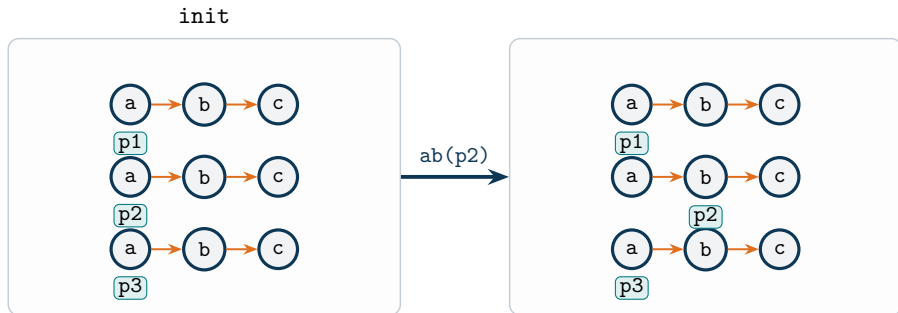


- **a**, **b**, and **c** are locations.
- A process moves between the locations by transitions **ab** and **bc**.

2.2.2 The Multitask ABC System

In a multitask system, multiple processes run concurrently.

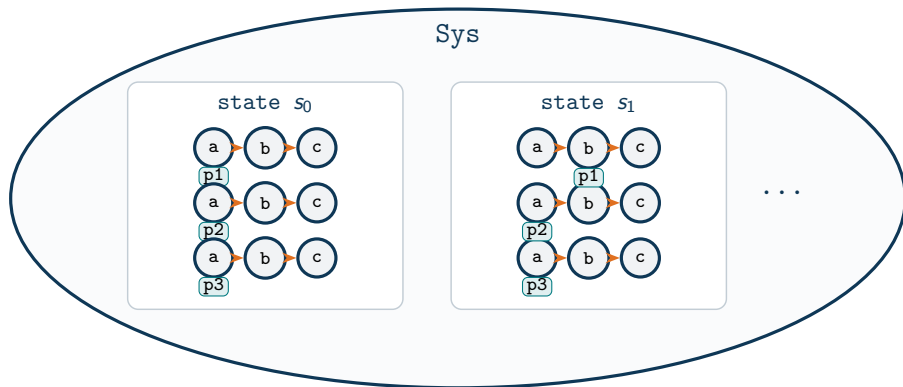
A system state records the current location of every process.



- Each process follows the same ABC transition system.
- A system state consists of the locations of all processes.
- A transition such as ab by $p2$ changes only the location of $p2$.

2.2.3 State space of the Multitask ABC System

We model the state space of such a multitask system by the sort `Sys` in the CafeOBJ specification.



Each system state is represented by an element of `Sys`.

We model such a multitask system as an OTS.

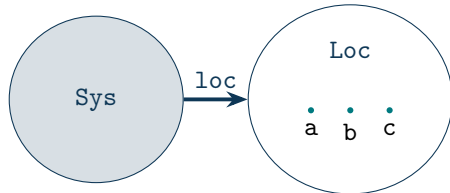
2.2.4 Observational transition system (OTS)

An OTS consists of observations, initial state(s), and transitions.

In an OTS, states are characterized by their observable values; their internal structure is abstracted away.

For the multitask ABC system:

- observation: `loc`;
- initial state: every process is at location `a`;
- transitions: `ab` and `bc`.



2.2.5 Specification of the ABC multitask system

The system module `ABC` is given as an OTS/CafeOBJ specification.

```
mod* ABC {  
  pr(LOC + PID)  
  *[Sys]*  
  op init : -> Sys  
  bop loc : Sys Pid -> Loc  
  bops ab bc : Sys Pid -> Sys  
  ops c-ab c-bc : Sys Pid -> Bool  
  var S : Sys .  
  vars I J : Pid .  
  eq loc(init,I) = a .  
  eq c-ab(S,I) = (loc(S,I) = a) .  
  ceq ab(S,I) = S if not c-ab(S,I) .  
  ceq loc(ab(S,I),J) =  
    (if I = J then b else loc(S,J) fi) if c-ab(S,I) .  
  eq c-bc(S,I) = (loc(S,I) = b) .  
  ceq bc(S,I) = S if not c-bc(S,I) .  
  ceq loc(bc(S,I),J) =  
    (if I = J then c else loc(S,J) fi) if c-bc(S,I) .  
}
```

ABC.cafe (module definition)

2.2.6 Data modules of ABC

The module `ABC` begins by importing the data modules used in the system specification.

```
mod* ABC {  
  pr(LOC + PID)  
  *[Sys]*  
  op init : -> Sys  
  bop loc : Sys Pid -> Loc  
  bops ab bc : Sys Pid -> Sys  
  ops c-ab c-bc : Sys Pid -> Bool  
  ...  
}
```

ABC.cafe (excerpt)

- `ABC` has the module import declaration `pr(LOC + PID)`.
- `pr` denotes a protecting import.
- A protecting import uses the imported modules without changing them.
- `LOC`: data module for values of discrete observations (locations).
- `PID`: data module for process identifiers.

2.2.7 Hidden sort of ABC

The hidden sort `Sys` denotes the state space, and `init` denotes an initial state.

```
mod* ABC {  
  pr(LOC + PID)  
  *[Sys]*  
  op init : -> Sys  
  bop loc : Sys Pid -> Loc  
  bops ab bc : Sys Pid -> Sys  
  ops c-ab c-bc : Sys Pid -> Bool  
  ...  
}
```

ABC.cafe (excerpt)

- `Sys` is the hidden sort representing the state space.
- `init` denotes an initial state.

2.2.8 Behavioral operations of ABC

Observations and transitions are declared as behavioral operations.

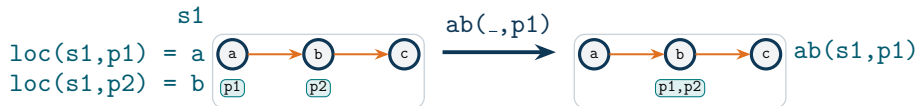
```
mod* ABC {  
  pr(LOC + PID)  
  *[Sys]*  
  op init :  -> Sys  
  bop loc :  Sys Pid -> Loc  
  bops ab bc : Sys Pid -> Sys  
  ops c-ab c-bc : Sys Pid -> Bool  
  ...  
}
```

ABC.cafe (excerpt)

- `loc` is an observation.
- `ab` and `bc` are transitions.

`loc(S,I)` means the location of process `I` in state `S`.

`ab(S,I)` means the state obtained by applying transition `ab` to process `I` in state `S`.



2.2.9 Initial-state equation of ABC

The first equation specifies the location of every process in the initial state.

```
mod* ABC {  
  ...  
  var S : Sys .  
  vars I J : Pid .  
  eq loc(init,I) = a .  
  ...  
}
```

ABC.cafe (excerpt)

- `I` denotes an arbitrary process identifier.
- `eq loc(init,I) = a .`
declares that every process is at location `a` in the initial state `init`.

2.2.10 Effective condition of a transition

Each transition in an OTS/CafeOBJ specification has an effective condition.

```
mod* ABC {  
  ...  
  ops c-ab c-bc : Sys Pid -> Bool  
  ...  
  eq c-ab(S,I) = (loc(S,I) = a) .  
  ...  
}
```

ABC.cafe (excerpt)

- $c\text{-}ab(S,I)$ is the effective condition of transition $ab(S,I)$.
- The `ops` declaration introduces $c\text{-}ab$ and $c\text{-}bc$; here we focus on $c\text{-}ab$.
- The equation defines its condition.
- Process I can apply transition ab only when I is at location a .

2.2.11 Effect of transition `ab` on observations

When `ab` is effective, it changes the observed location of process `I`.

```
mod* ABC {
```

ABC.cafe (excerpt)

```
...
```

```
ceq loc(ab(S,I),J) =  
  (if I = J then b else loc(S,J) fi)  
  if c-ab(S,I) .
```

```
eq c-bc(S,I) = (loc(S,I) = b) .
```

```
ceq bc(S,I) = S if not c-bc(S,I) .
```

```
ceq loc(bc(S,I),J) =  
  (if I = J then c else loc(S,J) fi)  
  if c-bc(S,I) .
```

```
...
```

```
}
```

$\text{loc}(\text{ab}(S,I), J)$ is the location of process `J` in the state obtained by applying `ab` to process `I` in state `S`.

- If $J = I$, $\text{loc}(\text{ab}(S,I), J) = b$.
- If $J \neq I$, $\text{loc}(\text{ab}(S,I), J) = \text{loc}(S, J)$: the location of `J` is unchanged from state `S`.

Thus, `ab` changes only the location of process `I`.

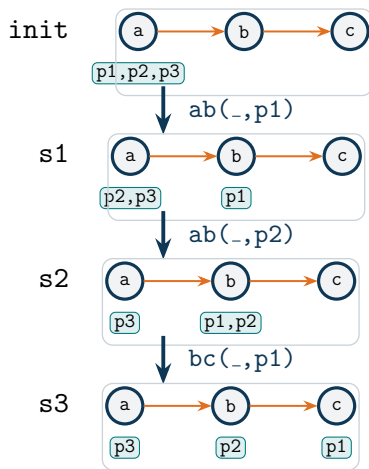
Transition `bc` is specified in the same way.

2.2.12 Execution trace of ABC

An opened module can define successive states and reduce their observations to execute a system trace.

```
open ABC .  
  ops p1 p2 p3 : -> Pid .  
  ops s1 s2 s3 : -> Sys .  
  eq (p1 = p2) = false .  
  eq (p1 = p3) = false .  
  eq (p2 = p3) = false .  
  eq s1 = ab(init, p1) .  
  eq s2 = ab(s1, p2) .  
  eq s3 = bc(s2, p1) .  
  red loc(s1,p1) . -- b  
  red loc(s1,p2) . -- a  
  red loc(s2,p1) . -- b  
  red loc(s2,p2) . -- b  
  red loc(s3,p1) . -- c  
  red loc(s3,p2) . -- b  
close
```

2.2.11.cafe (excerpt)

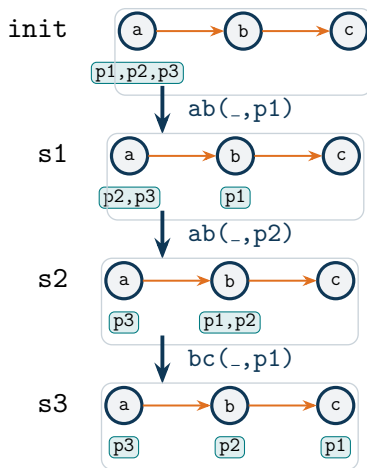


2.2.13 Execution trace of ABC

An opened module can define successive states and reduce their observations to execute a system trace.

```
CafeOBJ> in 2.2.11.cafe
```

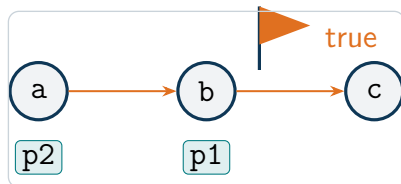
```
-- reduce in %ABC : (loc(s1,p1)):Loc  
(b):Loc  
-- reduce in %ABC : (loc(s1,p2)):Loc  
(a):Loc  
-- reduce in %ABC : (loc(s2,p1)):Loc  
(b):Loc  
-- reduce in %ABC : (loc(s2,p2)):Loc  
(b):Loc  
-- reduce in %ABC : (loc(s3,p1)):Loc  
(c):Loc  
-- reduce in %ABC : (loc(s3,p2)):Loc  
(b):Loc
```



2.2.14 The ABCf System: Mutual Exclusion

We now consider the **ABCf** system, which introduces a Boolean flag to control entry into location **b**.

- Location **b** represents a shared resource or critical section.
- At most one process may be at **b**.
- A Boolean flag enforces this restriction.
- Thus, **ABCf** is a simple model of mutual exclusion.



b: critical section

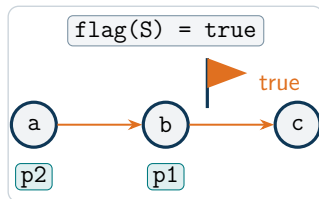
At most one process at **b**.

2.2.15 Flag extension ABCf

The module `ABCf` adds a Boolean observation `flag` that records whether location `b` is occupied.

```
mod* ABCf {  
  pr(LOC + PID)  
  *[Sys]*  
  op init :  -> Sys  
  bop loc :  Sys Pid -> Loc  
  bop flag : Sys -> Bool  
  bops ab bc : Sys Pid -> Sys  
  ops c-ab c-bc : Sys Pid -> Bool  
  ...  
}
```

ABCf.cafe (excerpt)



- `flag` is a Boolean observation.
- `true`: `b` is occupied; `false`: `b` is free.
- We redefine transition `ab` so that it allows entry only while the flag is lowered.

2.2.16 Initial observations of ABCf

ABCf adds `flag` as another observation of the system state. Thus, a state is observed by both process locations and the `flag` value.

```
mod* ABCf {  
  ...  
  var S : Sys .  
  vars I J : Pid .  
  eq loc(init,I) = a .  
  eq flag(init) = false .  
  ...  
}
```

ABCf.cafe (excerpt)

- `loc(init,I) = a` means that every process is initially at location `a`.
- `flag(init) = false` means that location `b` is initially free.

2.2.17 Equations for ab in ABCf

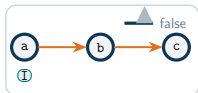
An effective ab transition moves a process to b and raises the occupancy flag.

```
mod* ABCf { ...
```

ABCf.cafe (excerpt)

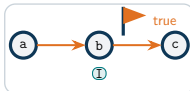
```
  eq c-ab(S,I) =  
    (loc(S,I) = a) and not flag(S) .  
  ceq ab(S,I) = S if not c-ab(S,I) .  
  ceq loc(ab(S,I),J) =  
    (if I = J then b else loc(S,J) fi)  
    if c-ab(S,I) .  
  ceq flag(ab(S,I)) = true  
    if c-ab(S,I) .
```

```
... }
```



I at a; flag = false

$ab(_, I)$



I at b; flag = true

- $c-ab(S,I)$ is true when I is at a and $flag(S)$ is false.
- If false, $ab(S,I) = S$; the state is unchanged.
- If effective, I moves to b ; other processes retain their locations, as before.
- $flag(ab(S,I)) = true$: the flag is raised because b is occupied.

2.2.18 Equations for bc in ABCf

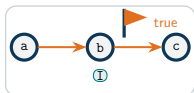
An effective bc transition moves a process to c and lowers the occupancy flag.

```
mod* ABCf { ...
```

ABCf.cafe (excerpt)

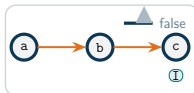
```
  eq c-bc(S,I) = (loc(S,I) = b) .  
  ceq bc(S,I) = S if not c-bc(S,I) .  
  ceq loc(bc(S,I),J) =  
    (if I = J then c else loc(S,J) fi)  
    if c-bc(S,I) .  
  ceq flag(bc(S,I)) = false  
    if c-bc(S,I) .
```

```
... }
```



I at b; flag = true

$bc(_, I)$



I at c; flag = false

- $c-bc(S,I)$ is true when process I is at b .
- If false, $bc(S,I) = S$; the state is unchanged.
- If effective, I moves from b to c ; other processes retain their locations.
- $flag(bc(S,I)) = false$: I leaves b , so b becomes free.

2.2.19 Ineffective transition in ABCf

A transition whose effective condition is false leaves the state unchanged.

```
open ABCf .
```

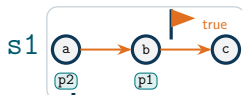
2.2.15.cafe (excerpt)

```
ops p1 p2 : -> Pid .
ops s1 s2 s3 s4 : -> Sys .
eq (p1 = p2) = false .
eq s1 = ab(init,p1) .
eq s2 = ab(s1,p2) .
eq s3 = bc(s2,p1) .
eq s4 = ab(s3,p2) .
red loc(s2,p2) . -- a
red loc(s4,p2) . -- b
close
```

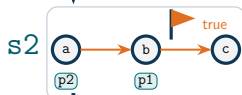
CafeOBJ> in 2.2.15.cafe

(a):Loc

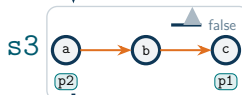
(b):Loc



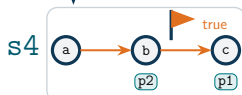
ab(-,p2)



bc(-,p1)



ab(-,p2)



In s1, $c-ab(s1,p2)$ is false because $flag(s1)$ is true.

Thus, $s2 = ab(s1,p2) = s1$.

2.3 Verification with proof scores

We verify mutual exclusion in `ABCf` as a state predicate over reachable states.

Mutual-exclusion property

No two distinct processes are simultaneously at location `b`.

```
eq inv1(S,I,J) =  
  ((loc(S,I) = b and loc(S,J) = b)  
   implies (I = J)) .
```

`inv1(S,I,J)` states that if both `I` and `J` are at location `b` in state `S`, then `I` and `J` are the same process.

Equivalently, by contraposition, two distinct processes cannot be simultaneously at `b`.

Definition. A state predicate P is *invariant* if $P(S)$ is true in every state S reachable from the initial state.

Our verification goal is to prove that `inv1` is invariant.

2.3.1 Evaluation of `inv1` in a reachable state

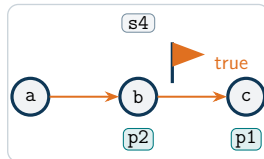
State `s4` from the preceding execution trace is a concrete reachable state.

```
open INV .
...
eq s1 = ab(init,p1) .
eq s2 = ab(s1,p2) .
eq s3 = bc(s2,p1) .
eq s4 = ab(s3,p2) .
red inv1(s4,p1,p2) .
red inv1(s4,p2,p2) .
close
```

ABCf/test.cafe (excerpt)

```
CafeOBJ> in ABCf/test.cafe
(true):Bool
(true):Bool
```

```
inv1(S,I,J) =
  ((loc(S,I)=b and loc(S,J)=b)
   implies (I=J))
```



- `inv1(s4,p1,p2)` reduces to `true`: the premise is false because `p1` is not at `b`.
- `inv1(s4,p2,p2)` reduces to `true`: its premise is true, and so is the conclusion `p2 = p2`.

Checking individual reachable states is not enough. We prove that `inv1` is invariant for arbitrary process identifiers by induction on state transitions.

2.3.2 Base step

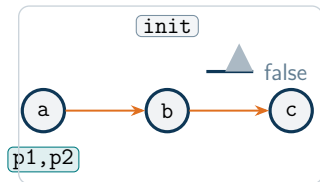
Proof module `INV` imports `ABCf` and defines `inv1`; the base step checks it in the initial state for arbitrary process identifiers.

```
open INV . --> inv1 init
ops p1 p2 : -> Pid .
red inv1(init,p1,p2) .
close
```

ABCf/inv1.cafe (excerpt)

```
CafeOBJ> in ABCf/inv1.cafe
(true):Bool
```

```
inv1(S,I,J) =
  ((loc(S,I)=b and loc(S,J)=b)
   implies (I=J))
```



- `p1` and `p2` are fresh constants, so they represent arbitrary elements of `Pid`.
- We reduce `inv1(init,p1,p2)`.
- The reduction result is `true`; therefore the base step holds for arbitrary process identifiers.

2.3.3 Induction step

For each transition τ , the induction step proves that `inv1` is preserved from an arbitrary reachable state. In `ABCf`, τ is `ab` or `bc`.

$$\text{inv1}(s, I, J) \implies \text{inv1}(\tau(s), I, J)$$

We introduce predicate `istep1` for the induction step.

```
op istep1 : Pid Pid -> Bool . ABCf/main.cafe (excerpt)
ops s s' : -> Sys .
eq istep1(I,J) =
  (inv1(s,I,J) implies inv1(s',I,J)) .
```

- `s`: an arbitrary reachable state.
- `s'`: the result state after applying a transition to `s`.
- `inv1(s,I,J)`: the induction hypothesis.

For example, a proof passage defines the result state by

$$\text{eq } s' = \text{ab}(s, p1) .$$

2.3.4 Proof passage for an induction step

We first try the induction step for transition `ab` without adding case-splitting equations.

```
open INV . --> inv1 ab illustrative proof passage
  ops p1 p2 p3 : -> Pid .
  eq s' = ab(s,p3) .
  red istep1(p1,p2) .
close
```

Intermediate reduction result:

```
((loc(ab(s,p3),p2) = b)
 and ((p1 = p2) and ...)) ...):Bool
```

- The reduction returns a term that is neither `true` nor `false`.
- The available equations are insufficient to reduce the term any further.
- Such a result is called an *intermediate term*.
- To continue, we add equations obtained by case-splitting.

2.3.5 Case-splitting on the effective condition

We first split the proof according to the effective condition $c\text{-}ab(s, p3)$.

Case 1: ineffective ab

```
open INV .  
  ops p1 p2 p3 : -> Pid .  
  eq s' = ab(s,p3) .
```

Illustrative case split

```
eq c-ab(s,p3) = false .
```

```
  red istep1(p1,p2) .  
close
```

Result: $(\text{true}) : \text{Bool}$

Case 2: effective ab

```
open INV .  
  ops p1 p2 p3 : -> Pid .  
  eq s' = ab(s,p3) .
```

```
eq loc(s,p3) = a .  
eq flag(s) = false .
```

```
  red istep1(p1,p2) .  
close
```

Result: intermediate term

```
(((((if (p3 = p2) then b else loc(s,p2) fi  
      ) = b)  
and ((p1 = p2) and ...)) ...)):Bool
```

The two Case 2 equations are the defining conditions of effective ab .

2.3.6 Further case-splitting

We inspect the intermediate term returned by the effective-**ab** case to find the next case-splitting condition.

```
... ((if (p3 = p2) then b else loc(s,p2) fi) = b)
```

Intermediate reduction result

```
and ( p1 = p2 and
```

```
((if (p3 = p1) then b else loc(s,p1) fi) = b) ...
```

CafeOBJ cannot determine whether $p1 = p2$ from the current equations. Therefore we add the undecided equation as a premise and split the proof passage.

- Case 1: $p1 = p2$
- Case 2: $(p1 = p2) = \text{false}$

2.3.7 Further case-splitting: $p1 = p2 / p1 \neq p2$

We next split the proof according to $p1 = p2$.

Case 1: $p1 = p2$

```
open INV .  
...  
eq loc(s,p3) = a .  
eq flag(s) = false .  
...  
eq p1 = p2 .  
...  
close
```

Illustrative case split

Result: `(true):Bool`

Case 2: $p1 \neq p2$

```
open INV .  
...  
eq loc(s,p3) = a .  
eq flag(s) = false .  
...  
eq (p1 = p2) = false .  
...  
close
```

Result: intermediate term

```
((((if (p3 = p1) then b else loc(s,p1) fi)  
= b) and ...)) ... : Bool
```

One branch is proved, while the other still returns an intermediate term and requires further case-splitting.

2.3.8 False proof passage

Repeated case-splitting may produce a proof passage whose reduction result is `false`.

```
open INV .  
  ops p1 p2 p3 : -> Pid .  
  eq s' = ab(s,p3) .  
  eq loc(s,p3) = a .  
  eq flag(s) = false .  
  eq (p1 = p2) = false .  
  eq p2 = p3 .  
  eq (p1 = p3) = false .  
  eq loc(s,p1) = b .  
  red istep1(p1,p2) .  
close
```

False proof passage during construction

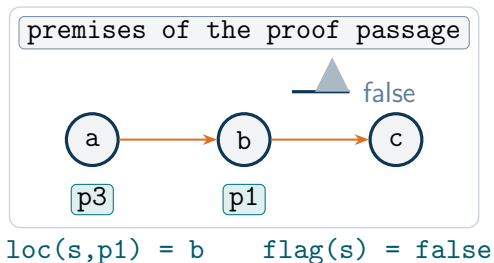
Reduction result:
`(false):Bool`

A false reduction result does not necessarily mean that the mutual-exclusion property is false.

Case-splitting may generate proof passages corresponding to cases that need not be considered in the induction over reachable states.

2.3.9 Analyzing a false proof passage

Let us inspect what state is described by the premises of this false proof passage.



The premises describe a state in which **p1** is at **b** while the flag is **false**.

Such a state cannot be reached from the initial state of our **ABCf** model.

Therefore, this false proof passage represents an unreachable case.

In the proof score method, a proof passage believed to represent an unreachable case is discharged by introducing an appropriate lemma.

False Proof Passages and Reachable States

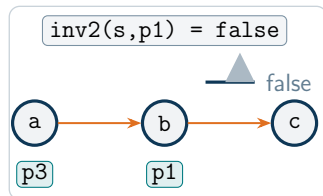
A false proof passage can arise from an unreachable state that satisfies `inv1`.



2.3.10 Lemma candidate `inv2`

To discharge this proof passage, we introduce a lemma candidate `inv2`.

```
op inv2 : Sys Pid -> Bool
eq inv2(S,I) =
  ((loc(S,I) = b) implies flag(S)) .
```



For the preceding proof passage, `loc(s,p1)=b` and `flag(s)=false`; therefore `inv2(s,p1)` is `false`. Thus, `inv2` excludes exactly the kind of state represented by this false proof passage.

2.3.11 Use of `inv2` in the proof passage

By adding the lemma candidate `inv2` to the premise of the proof passage, the reduction returns `true`.

```
open INV .
  ops p1 p2 p3 : -> Pid .
  eq s' = ab(s,p3) .
  eq loc(s,p3) = a .
  eq flag(s) = false .
  eq (p1 = p2) = false .
  eq p2 = p3 .
  eq (p1 = p3) = false .
  eq loc(s,p1) = b .
  red inv2(s,p1) implies istep1(p1,p2) .
close
```

ABCf/inv1.cafe (excerpt)

In this proof passage, `inv2(s,p1)` is `false` because `loc(s,p1)=b` and `flag(s)=false`.

Therefore the premise of

$$\text{inv2}(s,p1) \Rightarrow \text{istep1}(p1,p2)$$

is false.

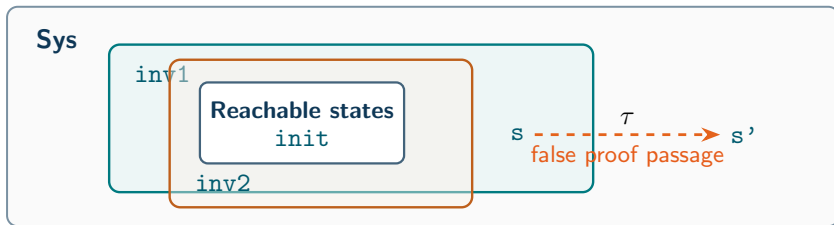
Reduction result:

`(true):Bool`

The other false proof passages for `inv1` are also discharged with `inv2` in the same way. As a result, all proof passages for `inv1` reduce to `true`.

How inv2 Restricts the Induction Premise

The invariants `inv1` and `inv2` are used together; `inv2` excludes the unreachable state responsible for the false passage.



```
inv2(s,p1) implies istep1(p1,p2)
= inv2(s,p1) implies (inv1(s,p1,p2) implies inv1(s',p1,p2))
= (inv2(s,p1) and inv1(s,p1,p2)) implies inv1(s',p1,p2)
```

2.3.12 Induction proof for `inv2`

The remaining task is to prove that `inv2` is invariant.

```
open INV .  
  ops p1 p2 p3 :  -> Pid .  
  eq s' = bc(s,p3) .  
  eq loc(s,p3) = b .  
  eq (p1 = p3) = false .  
  eq loc(s,p1) = b .  
  eq flag(s) = true .  
  red inv1(s,p1,p3) implies istep2(p1) .  
close
```

ABCf/inv2.cafe (excerpt)

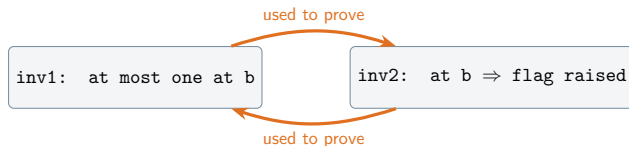
Reduction result:

`(true):Bool`

- We prove `inv2` invariant by induction in the same way.
- While one proof passage for `inv2` needs `inv1` as a lemma, all proof passages for `inv2` reduce to `true`.

2.3.13 Invariance of `inv1` and `inv2`

We use `inv2` to prove `inv1` and use `inv1` to prove `inv2`. This may look circular at first, but it is not circular reasoning.



The proof actually establishes the stronger invariant `inv1` $\wedge$ `inv2`. Its induction step is

$$\text{inv1}(s, p_1, p_2) \wedge \text{inv2}(s, p_3) \Longrightarrow \text{inv1}(s', p_1, p_2) \wedge \text{inv2}(s', p_3).$$

It decomposes into two goals:

$$\text{inv1}(s, p_1, p_2) \wedge \text{inv2}(s, p_3) \Longrightarrow \text{inv1}(s', p_1, p_2),$$

$$\text{inv1}(s, p_1, p_2) \wedge \text{inv2}(s, p_3) \Longrightarrow \text{inv2}(s', p_3).$$

The `inv1` proof passages establish the first goal; the `inv2` proof passages establish the second. Both may assume `inv1(s)` and `inv2(s)` together as the induction hypothesis for the conjunction. Thus their mutual use is legitimate.

2.3.14 Completion of the ABCf proof

The complete proof score establishes the main invariant `inv1` together with one auxiliary lemma `inv2`.

- `inv2` discharges the false proof passage shown above and a second similar false passage for `inv1`.
- `inv2` is itself proved invariant.
- One false proof passage for `inv2` is discharged using `inv1`.
- Finally, all proof passages return `true`; therefore the proof score is complete.

Proof files: `ABCf/inv1.cafe`, `ABCf/inv2.cafe`

Complete proof score

- Main invariant: `inv1`
- Auxiliary lemma: `inv2` (one)
- Final proof passages: 21
 - 13 for `inv1`
 - 8 for `inv2`
- All 21 reductions: `true`

Outline

1. Introduction
2. The OTS/CafeOBJ Method Overview
- 3. Modeling Multitask Real-Time Systems in OTS/CafeOBJ Specifications**
4. Case study: Fischer's protocol
5. Case study: autonomous vehicle control system
6. Conclusion

3 Modeling Multitask Real-Time Systems in OTS/CafeOBJ Specifications

This section introduces a way to model real-time systems in the OTS/CafeOBJ framework; detailed definitions appear in [6, 7].

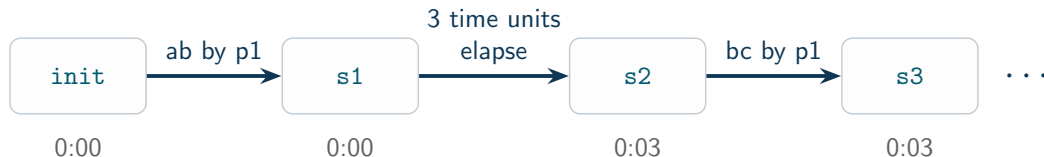
3.1 Elapsed-time observation and time-advancing transition

3.2 Time constraints with lower and upper bounds

3.1 Elapsed-time observation and time-advancing transition

A real-time system can be modeled as a state-transition system with discrete and continuous (time-advancing) transitions.

- Discrete transitions are the transitions used so far, such as `ab` and `bc`.
- A continuous, or time-advancing, transition represents the passage of time.
- The system state also has elapsed-time information.



3.1.1 Real-time extension of an OTS

CafeOBJ provides the built-in module `RAT` for rational numbers; `now` and `tick` will then be used to represent elapsed time and time advancement.

- `RAT` provides rational numbers, including `0`, `2`, `-3`, and `4/5`, and operations such as `+`, `*`, and `<=`.
- The rational numbers are dense: between any two distinct rational numbers, there is another rational number; thus, positive rational values can be arbitrarily small.
- `now`: the elapsed-time observation.
- `tick`: the time-advancing transition, parameterized by the elapsed duration.

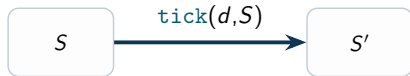
Detailed definitions of real-time OTSs appear in [6, 7].

3.1.2 Real-time extension ABCt

We define **ABCt** as a real-time extension of the ABC system by adding elapsed time and time advancement.

```
mod* ABCt {  
  pr(LOC + PSET + RAT)  
  *[Sys]*  
  ...  
  bop now : Sys -> Rat  
  ...  
  bop tick : Rat Sys -> Sys  
  ...  
}
```

ABCt.cafe (excerpt)



$$\text{now}(S) = t$$

$$\text{now}(S') = t + d$$

- $\text{now}(S)$: the elapsed time in state S .
- $\text{tick}(T, S)$: the state after T time units from state S .

3.1.3 Discrete and continuous behavior

In `ABCT`, discrete transitions are instantaneous, while `tick` advances time without changing discrete observations.

- Every discrete transition τ preserves elapsed time:

$$\text{now}(\tau(S)) = \text{now}(S).$$

- `tick` changes only `now` in this example and preserves every discrete observation o :

$$o(\text{tick}(T, S)) = o(S).$$

3.1.4 ABCt specification

The real-time extension adds `now` and `tick`, together with equations for them.

```
bop now : Sys -> Rat
bop tick : Rat Sys -> Sys

eq now(init) = 0 .
eq now(ab(S,I)) = now(S) .
eq now(bc(S,I)) = now(S) .

eq c-tick(D,S) = (0 <= D) .
ceq tick(D,S) = S
  if not c-tick(D,S) .
ceq loc(tick(D,S),J) = loc(S,J)
  if c-tick(D,S) .
ceq now(tick(D,S)) = now(S) + D
  if c-tick(D,S) .
```

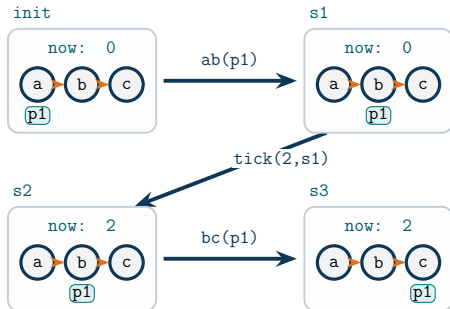
ABCt.cafe (excerpt)

3.1.5 Timed execution in CafeOBJ

The following CafeOBJ code executes an example trace of [ABCt](#).

```
open ABCt .  
  op p1 : -> Pid .  
  ops s1 s2 s3 : -> Sys .  
  
  eq s1 = ab(init,p1) .  
  eq s2 = tick(2,s1) .  
  eq s3 = bc(s2,p1) .  
  
  red loc(s3,p1) .  
  red now(s3) .  
  
close
```

3.1.7.cafe (excerpt)



3.1.6 Timed execution in CafeOBJ

The following output is obtained from the `ABCt` execution.

```
CafeOBJ> in 3.1.7.cafe
```

```
-- reduce in %ABCt :
```

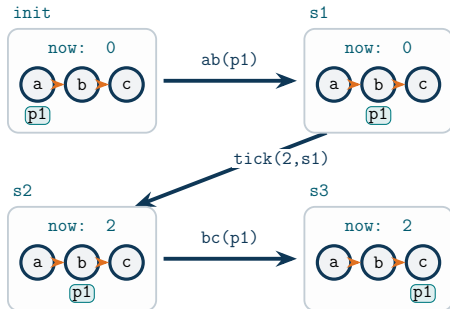
```
  (loc(s3,p1)):Loc
```

```
(c):Loc
```

```
-- reduce in %ABCt :
```

```
  (now(s3)):Rat
```

```
(2):NzNat
```



3.2 Time constraints

In real-time systems, it is often necessary to constrain when a transition may or must occur.

In ABC_t , for example, we may require bc to occur between three and five time units after a process enters location b .



We now show how such time constraints are modeled in a real-time OTS/CafeOBJ specification.

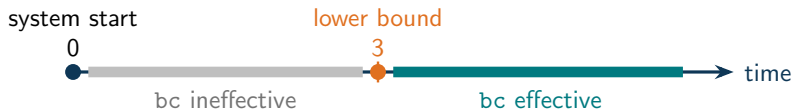
3.2.1 Lower bounds

First, suppose that we want to prevent `bc` from being taken during the first three time units after the system starts.

This requirement can be modeled by adding the time constraint to the effective condition:

$$c\text{-}bc(S, I) = (3 \leq \text{now}(S))$$

- While `now(S) < 3`, `bc` cannot be taken.
- When `now(S) >= 3`, `bc` may be taken.



The lower bound is incorporated directly into the effective condition.

3.2.1.1 Relative-time lower bound

Now suppose that we want to prevent `bc` from being taken until three time units after the process enters location `b`.

To express a bound relative to the time at which the process enters `b`, we record the corresponding time with observation `l`.

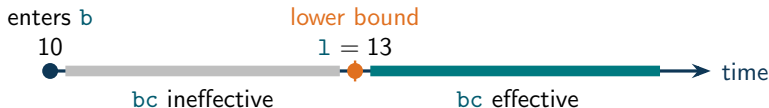
$$l(ab(S, I), I) = \text{now}(S) + 3$$

When process `I` enters `b`, `l` records the time at which `bc` may become available for `I`.

The effective condition uses that recorded time:

$$c - bc(S, I) = (l(S, I) \leq \text{now}(S)).$$

Thus, `bc` cannot be taken before `l(S, I)` and may be taken when that time is reached.



3.2.1.2 ABCt2 specification with a lower bound

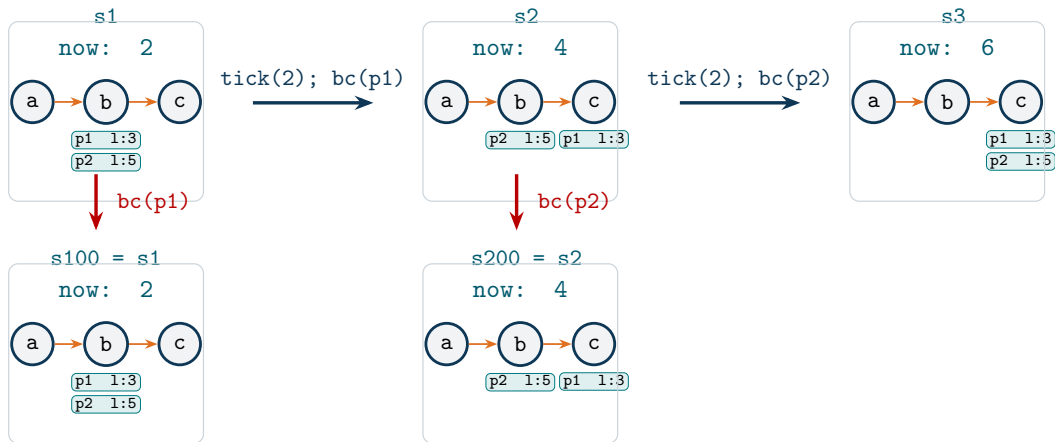
The module records a process-specific lower-bound time and includes it in the effective condition of `bc`.

```
mod* ABCt2 {  
  ...  
  bop l : Sys Pid -> Rat  
  ...  
  var S : Sys .  
  vars I J : Pid .  
  eq l(init,I) = 0 .  
  ...  
  ceq l(ab(S,I),J) =  
    (if I = J then now(S) + 3 else l(S,J) fi)  
    if c-ab(S,I) .  
  ...  
  eq c-bc(S,I) = (loc(S,I) = b)  
    and (l(S,I) <= now(S)) .  
  ...  
}
```

ABCt2.cafe (excerpt)

3.2.1.3 Lower-bound execution trace

The trace includes effective transitions after their lower bounds and ineffective transitions attempted too early.



3.2.1.4 Definition of the lower-bound trace

The following equations define the states shown on the previous slide. States `s1`, `s2`, and `s3` form the successful path, while `s100` and `s200` represent early attempts to take `bc`.

```
open ABCt2 .
  ops p1 p2 : -> Pid .
  ops s1 s2 s3 s100 s200 : -> Sys .
  eq (p1 = p2) = false .
  eq s1 = ab(tick(2, ab(init, p1)), p2) .
  eq s100 = bc(s1, p1) .          -- failed
  eq s2 = bc(tick(2, s1), p1) .
  eq s200 = bc(s2, p2) .         -- failed
  eq s3 = bc(tick(2, s2), p2) .
  ...
```

3.2.1.4.cafe (excerpt)

3.2.1.5 Reduction results for the lower-bound trace

The following reductions are obtained for the states defined on the previous slide.

CafeOBJ> in 3.2.1.4.cafe

```
-- reduce in %ABCT2 : (now(s1)):Rat
(2):NzNat
-- reduce in %ABCT2 : (loc(s100,p1)):Loc
(b):Loc
-- reduce in %ABCT2 : (loc(s2,p1)):Loc
(c):Loc
-- reduce in %ABCT2 : (loc(s200,p2)):Loc
(b):Loc
-- reduce in %ABCT2 : (loc(s3,p2)):Loc
(c):Loc
```

Output from 3.2.1.4.cafe (excerpt)

3.2.2 Upper bounds

Suppose a process has entered location `b`; we want to require it to take `bc` within five time units.

In a single-process setting, assume that the process enters `b` at time 0. A naive condition such as

$$c - bc(S) = (\text{now}(S) \leq 5)$$

says only that `bc` may be taken while `now(S) ≤ 5`. It does not say that `bc` must be taken.



Therefore, an upper bound cannot be modeled only by the effective condition of `bc`.

3.2.2.1 Restricting time advancement

The effective condition of `tick` can restrict how far time is allowed to advance.

$$\text{c-tick}(D, S) = (0 \leq D) \text{ and } (\text{now}(S) + D \leq 100)$$

With this condition, `tick` may advance time only if the resulting elapsed time does not exceed 100. Thus, time cannot advance beyond 100.

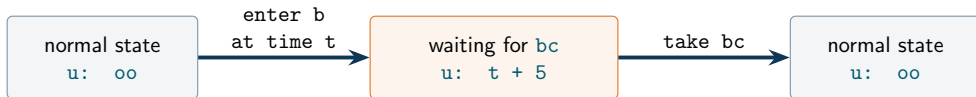


3.2.2.2 A state-dependent upper bound

A fixed bound such as 100 would eventually prevent time from advancing forever; that is not the behavior we want.

We need an upper bound while waiting for `bc`, but no upper bound after `bc` has been taken. We therefore introduce an observation `u` that records the current upper-bound time.

$$(0 \leq D) \text{ and } (\text{now}(S) + D \leq u(S))$$



The module `TIMEVAL` extends the rational numbers with infinity ∞ ; ∞ is greater than every rational number.

3.2.2.3 A single-process upper bound

For simplicity, let us first consider a single-process system.

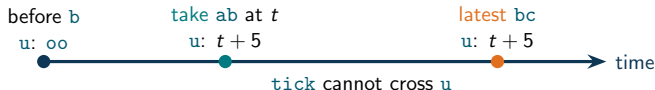
- Initially, there is no upper bound: $u = \infty$.
- Taking `ab` at time t sets $u = t + 5$.
- Until `bc` is taken, `tick` may advance time only up to u .
- Taking `bc` restores $u = \infty$.

$$\text{eq } u(\text{init}) = \infty .$$

$$\text{eq } u(\text{ab}(S)) = \text{now}(S) + 5 .$$

$$\text{eq } \text{c-tick}(D, S) = (0 \leq D) \\ \text{and } (\text{now}(S) + D \leq u(S)) .$$

$$\text{eq } u(\text{bc}(S)) = \infty .$$

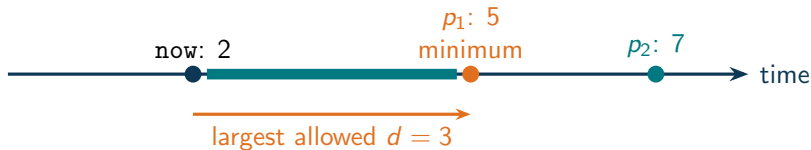


This explains the upper-bound mechanism for one process. We next consider a multitask system.

3.2.2.4 From one process to multiple processes

Now consider a multitask system with different upper bounds for different processes.

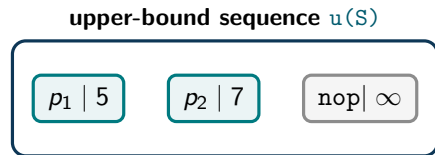
Suppose `now` = 2, the upper bound of `p1` is 5, and the upper bound of `p2` is 7.



The transition `tick` advances the global time of the entire system. A single `tick` must therefore respect the upper bounds of all processes. The earliest upper bound is 5, so the largest allowed duration is 3.

3.2.2.5 Collecting upper bounds

To constrain the global `tick`, we need the minimum of the upper bounds of all processes. We therefore store process-specific upper bounds in observation `u(S)`.



minimum = 5

We define module `PSET`, which specifies process-specific upper bounds as an ordered sequence of process/time pairs.

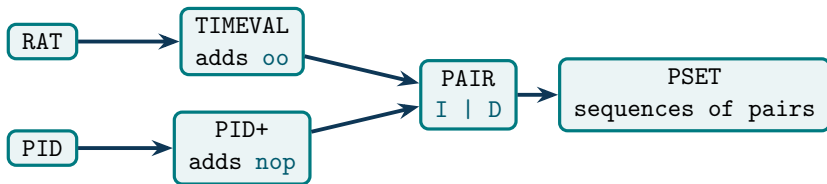
- `I | D`: process `I` with upper-bound time `D`.
- `nop | ∞`: a dummy pair denoting no upper limit.
- `min`: takes a `PSet` value containing the processes' upper limits and returns the minimum value.

$$\text{c-tick}(D, S) = (0 \leq D) \text{ and } (\text{now}(S) + D \leq \min(u(S)))$$

The global `tick` can advance time only up to the earliest process deadline.

3.2.2.6 Auxiliary data types for upper bounds

Four auxiliary modules provide the data representation for the terms stored in $u(S)$.



- **TIMEVAL**: rational numbers extended with infinity `oo`, which is greater than every rational number.
- **PID+**: process identifiers extended with `nop`.
- **PAIR**: pairs `I | D`.
- **PSET**: stores process/deadline pairs.

In **PSET**, a term of sort **PSet** is a sequence of process/upper-bound pairs, such as

`(p1 | 5) (p2 | 7) (nop | oo)`.

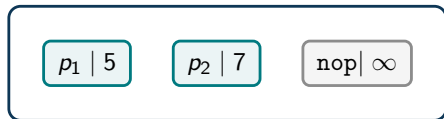
3.2.2.7 Operations on PSET

Module **PSET** defines operations for finding the minimum upper bound and removing the pair for a specified process.

```
mod PSET {
  ...
  op min : Pset -> Rat+
  op remove : Pid+ Pset -> Pset
  ...
  eq min((nop | oo)) = oo .
  eq min((I | D) PS) = min(D,min(PS)) .
  eq remove(I,(nop | oo)) = (nop | oo) .
  eq remove(I,(J | D) PS) =
    (if I = J then PS else
     (J | D) remove(I,PS) fi) .
  ... }
```

PSET.cafe (excerpt)

upper-bound sequence $u(S)$



minimum = 5

$\min(u(S)) = 5$

$\text{remove}(p_1, u(S)) =$
 $(p_2 \mid 7) (\text{nop} \mid \infty)$

3.2.2.8 Updating the upper-bound set

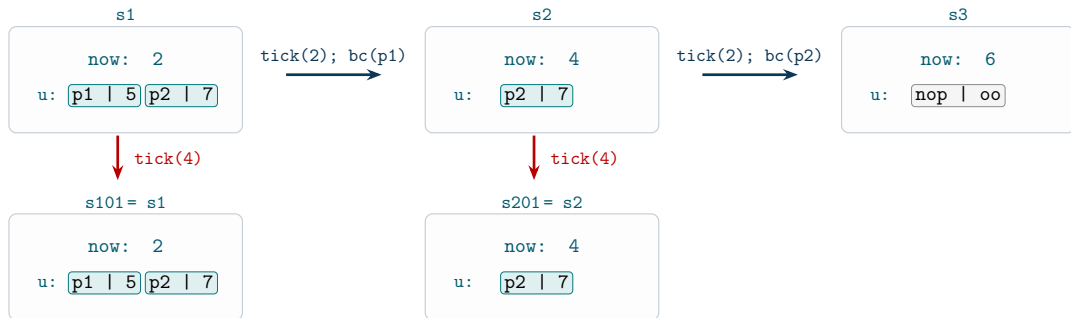
Transition `ab` adds a process deadline, and transition `bc` removes it.

```
mod* ABCt2 {
  pr(LOC + PSET + TIMEVAL)
  ...
  bop u : Sys -> Pset
  eq u(init) = (nop | oo) .
  ceq u(ab(S,I)) = (I | now(S) + 5) u(S)
    if c-ab(S,I) .
  ceq u(bc(S,I)) = remove(I,u(S))
    if c-bc(S,I) .
  eq c-tick(D,S) = (0 <= D) and (now(S) + D <= min(u(S))) .
  ... }
```

ABCt2.cafe (excerpt)

3.2.2.9 Upper-bound execution trace

Successful transitions follow the horizontal path; failed `tick(4)` attempts leave the state unchanged.



3.2.2.10 Definition of the upper-bound trace

The following equations define the successful path and the two failed time-advancing attempts.

```
open ABCt2 .  
  ops p1 p2 : -> Pid .  
  ...  
  eq (p1 = p2) = false .  
  eq s1 = ab(tick(2,ab(init,p1)),p2) .  
  eq s101 = tick(4,s1) . -- failed  
  eq s2 = bc(tick(2,s1),p1) .  
  eq s201 = tick(4,s2) . -- failed  
  eq s3 = bc(tick(2,s2),p2) .  
  ...
```

3.2.2.11.cafe (excerpt)

3.2.2.11 Reduction results for the upper-bound trace

A failed `tick` does not advance time, while the successful path reaches times 4 and 6.

CafeOBJ> in 3.2.2.11.cafe

```
-- reduce in %ABCT2 : (now(s101)):Rat
(2):NzNat
-- reduce in %ABCT2 : (now(s2)):Rat
(4):NzNat
-- reduce in %ABCT2 : (now(s201)):Rat
(4):NzNat
-- reduce in %ABCT2 : (now(s3)):Rat
(6):NzNat
```

Output from 3.2.2.11.cafe (excerpt)

The reductions confirm that the failed `tick` attempts do not advance elapsed time.

Outline

1. Introduction
2. The OTS/CafeOBJ Method Overview
3. Modeling Multitask Real-Time Systems in OTS/CafeOBJ Specifications
- 4. Case study: Fischer's protocol**
5. Case study: autonomous vehicle control system
6. Conclusion

4 Case study: Fischer's protocol

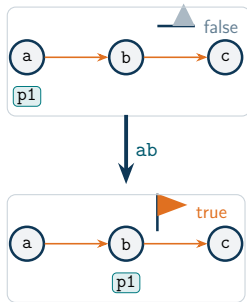
This case study combines the lower- and upper-bound mechanisms of Section 3 in a concrete multitask real-time specification.

4.1 OTS/CafeOBJ specification for Fischer's protocol

4.2 Verification for the mutual exclusion property of Fischer's protocol

4.1 OTS/CafeOBJ specification for Fischer's protocol

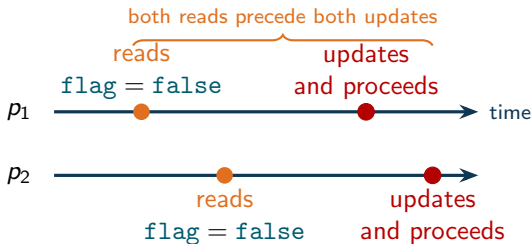
ABCf system



Transition **ab** tests the flag, moves the process to **b**, and sets the flag to **true** in one atomic action.

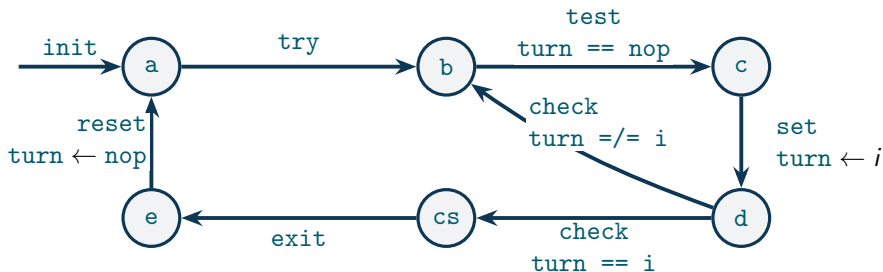
Fischer's protocol separates the test and update into two transitions, **test** and **set**.

However, if the test and update are separate, mutual exclusion is no longer guaranteed.



4.1.1 Fischer's protocol

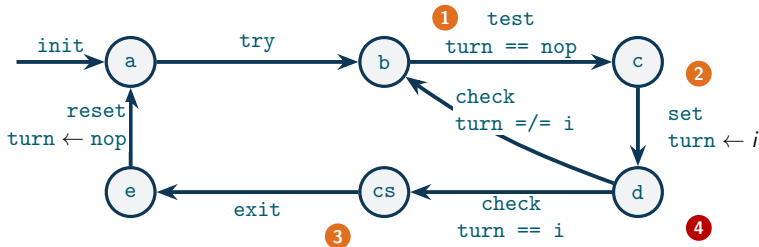
Fischer's protocol uses a global shared variable and real-time constraints to guarantee that at most one process is in location `cs`.



- `turn` is a global shared variable: `turn == i` means that process `i` is assigned, while `turn == nop` means that no process is assigned.
- `test` and `set` are separate transitions, not one atomic transition.

4.1.2 Why are time constraints needed?

Without timing constraints, separate `test` and `set` transitions can lead to two processes entering the critical section.



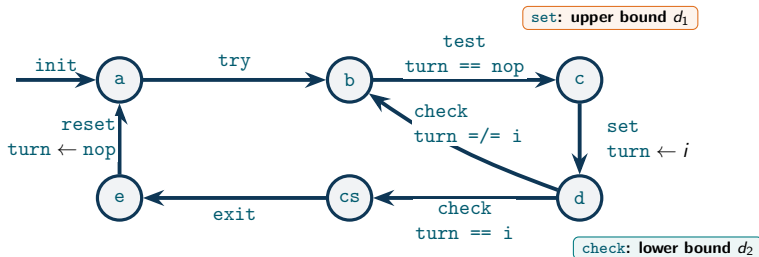
1. p_1 and p_2 both take `test` while `turn == nop` and reach c .
2. p_1 takes `set`: `turn` becomes p_1 , and p_1 moves from c to d .

3. p_1 takes `check` and reaches cs , while p_2 remains at c .
4. Without timing constraints, p_2 may later take `set`, overwrite `turn`, and eventually reach cs .

Fischer's protocol prevents this by introducing timing constraints.

4.1.3 Time constraints in Fischer's protocol

Fischer's protocol uses two timing constraints.



Upper bound d_1

After **test** moves a process to **c**, **set** has upper bound d_1 .

Lower bound d_2

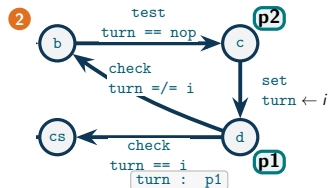
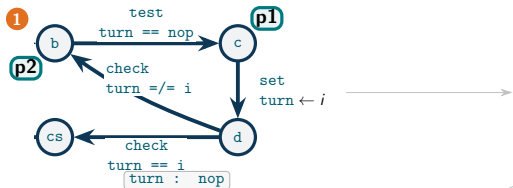
After **set** moves a process to **d**, **check** has lower bound d_2 .

check cannot be taken until d_2 time units have elapsed.

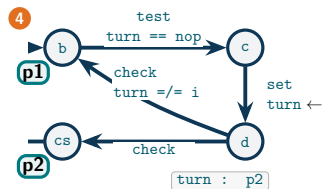
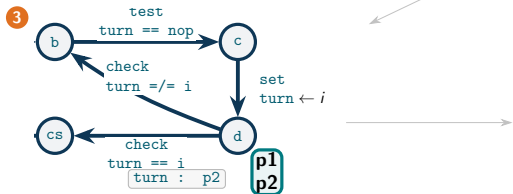
With $d_1 < d_2$, the protocol guarantees mutual exclusion.

4.1.4 Why does $d_1 < d_2$ matter?

The condition $d_1 < d_2$ guarantees mutual exclusion.



$p1$ must wait d_2 before check;
 $p2$ has upper bound d_1 for set.



Since $d_1 < d_2$, $p2$:set occurs before $p1$:check;
 $turn$ becomes $p2$.

4.1.5 FISCHER as a real-time OTS

The module declares the protocol observations, six discrete transitions, and the time-advancing transition `tick`.

```
*[ Sys ]*
op init : -> Sys
bop turn : Sys -> Pid+
bop loc  : Sys Pid -> Loc
bop now  : Sys -> Rat
bop l    : Sys Pid -> Rat
bop u    : Sys -> Pset
```

Fischer/main.cafe (excerpt)

```
bop try    : Sys Pid -> Sys
bop test   : Sys Pid -> Sys
bop set    : Sys Pid -> Sys
bop check  : Sys Pid -> Sys
bop exit   : Sys Pid -> Sys
bop reset  : Sys Pid -> Sys
bop tick   : Rat Sys -> Sys
```

`l(S,I)` is process-specific; the single observation `u(S)` contains the upper-bound pairs of all processes.

4.1.6 Initial state

Initially every process is at [a](#), elapsed time is zero, and no upper bound is present.

`init`

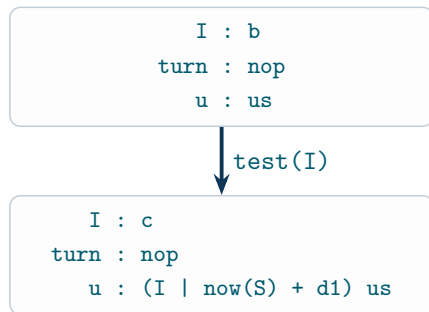
```
now : 0
turn : nop
loc(I) : a
l(I) : 0
u : nop | oo
```

```
eq turn(init) = nop .
eq loc(init,I) = a .
eq now(init) = 0 .
eq l(init,I) = 0 .
eq u(init) = (nop | oo) .
```

Fischer/main.cafe (excerpt)

4.1.7 Upper bound created by test

When process **I** executes **test** and enters **c**, an upper-bound pair is added to **u**.



```
eq c-test(S,I) =
  (loc(S,I) = b and turn(S) = nop) .
```

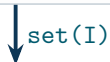
Fischer/main.cafe (excerpt)

```
ceq u(test(S,I)) =
  (I | now(S) + d1) u(S)
if c-test(S,I) .
```

4.1.8 Lower bound created by set

When process **I** executes **set** and enters **d**, its upper-bound pair is removed from **u**, and a lower bound for **check** is recorded in **l**. **turn** is set to **I**.

```
I : c  
turn : turn(S)  
l(I) : l(S,I)  
u : us (I | x) us'
```



```
I : d  
turn : I  
l(I) : now(S) + d2  
u : us us'
```

```
ceq loc(set(S,I),J) =  
  (if I = J then d else loc(S,J) fi)  
  if c-set(S,I) .
```

Fischer/main.cafe (excerpt)

```
ceq l(set(S,I),J) =  
  (if I = J then now(S) + d2  
   else l(S,J) fi) if c-set(S,I) .
```

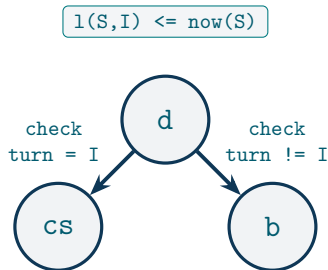
```
ceq u(set(S,I)) = remove(I,u(S)) if c-set(S,I) .
```

```
ceq turn(set(S,I)) = I if c-set(S,I) .
```

4.1.9 Lower bound on check

A process at d cannot take **check** until its recorded lower-bound time has been reached.

```
eq c-check(S,I) = Fischer/main.cafe (excerpt)  
  (loc(S,I) = d and  $l(S,I) \leq \text{now}(S)$ ) .  
  
ceq loc(check(S,I),J) =  
  (if  $I = J$  then  
    (if  $\text{turn}(S) = I$  then cs else b fi)  
    else loc(S,J) fi) if c-check(S,I) .  
  
ceq l(check(S,I),J) =  
  (if  $I = J$  then 0 else  $l(S,J)$  fi)  
  if c-check(S,I) .
```



4.2 Verification for the mutual exclusion property of Fischer's protocol

We now verify Fischer's mutual exclusion property using the proof-score method introduced in Section 2.

```
eq inv1(S,I,J) =  
  (loc(S,I) = cs and loc(S,J) = cs  
   implies I = J) .  
  
eq istep1(I,J) =  
  inv1(s,I,J) implies inv1(s',I,J) .
```

Fischer/main.cafe (excerpt)

- $\text{inv1}(S,I,J)$ states that if I and J are both at cs , then they are the same process.
- Goal: inv1 holds for all process pairs in every reachable state.

4.2.1 A representative false proof passage

During the proof for transition `check`, case-splitting produces the following passage.

```
open ISTEP .  
  ops i j k : -> Pid .  
  eq i = k .  
  eq (j = k) = false .  
  eq loc(s,j) = cs .  
  eq loc(s,k) = d .  
  eq (l(s,k) <= now(s)) = true .  
  eq turn(s) = k .  
  eq s' = check(s,k) .  
  red istep1(i,j) .  
close
```

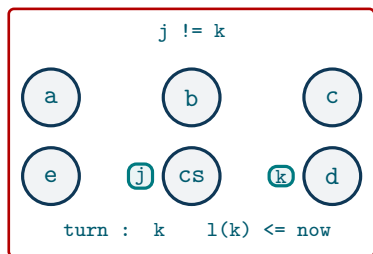
Illustrative false proof passage

Reduction result:

`(false):Bool`

4.2.2 The false passage describes an unreachable state

The premises describe two different processes in a state that cannot be reached from `init`.



unreachable from `init`

- Process `j` reached `cs` through `check` under the Fischer timing constraints.
- Under $d_1 < d_2$, another process cannot have remained at `c` long enough to overwrite `turn` and later reach this configuration.
- Thus the combination `j` at `cs`, `k` at `d`, and `turn=k` is unreachable.

4.2.3 Lemma `inv2`

The Fischer-specific lemma `inv2` excludes the unreachable combination in the preceding proof passage.

```
eq inv2(S,I,J) =  
  ((loc(S,I) = cs or loc(S,I) = e)  
   and not(I = J)  
   implies turn(S) = I  
    and not(loc(S,J) = c)) .
```

Fischer/main.cafe (excerpt)

If `I` is at `cs` or `e`, then `turn` is `I` and every different process `J` is not at `c`.

```
open FISCHER-PROOF-ENV .  
  ops i j k : -> Pid .  
  eq loc(s,k) = d .  
  eq (l(s,k) <= now(s)) = true .  
  eq i = k .  
  eq (j = k) = false .  
  eq turn(s) = k .  
  eq s' = check(s,k) .  
  red inv2(s,j,i)  
    implies istep1(i,j) .  
close
```

Fischer/inv1.cafe (excerpt)

Reduction result:
`(true):Bool`

4.2.4 Completion of the Fischer proof

The proof uses four invariant lemmata and 203 system-level proof passages; every passage returns `true`.

```
eq inv2(S,I,J) =  
  ((loc(S,I) = cs or loc(S,I) = e)  
   and not(I = J) implies turn(S) = I  
   and not(loc(S,J) = c)) .  
eq inv3(S,I,J) =  
  (loc(S,I) = d and turn(S) = I  
   and loc(S,J) = c and not(I = J)  
   implies get(J,u(S)) < l(S,I)) .  
eq inv4(S,I) =  
  (loc(S,I) = c implies  
   get(I,u(S)) < now(S) + d2) .  
eq inv5(S,I) = now(S) <= get(I,u(S)) .
```

Fischer/main.cafe (excerpt)

Proof files: Fischer/inv1.cafe ... Fischer/inv5.cafe

Outline

1. Introduction
2. The OTS/CafeOBJ Method Overview
3. Modeling Multitask Real-Time Systems in OTS/CafeOBJ Specifications
4. Case study: Fischer's protocol
- 5. Case study: autonomous vehicle control system**
6. Conclusion

5 Case study: autonomous vehicle control system

This case study specifies an autonomous vehicle control system and verifies that vehicles do not collide in any reachable state.

Unlike Fischer's protocol, this example combines continuous vehicle dynamics with discrete pedal-action transitions.

5.1 OTS/CafeOBJ specification for autonomous vehicle control system

5.2 Verification of the safety property

5.1 OTS/CafeOBJ specification for autonomous vehicle control system

We model a group of autonomous vehicles with continuous position x and velocity v , and discrete pedal states.



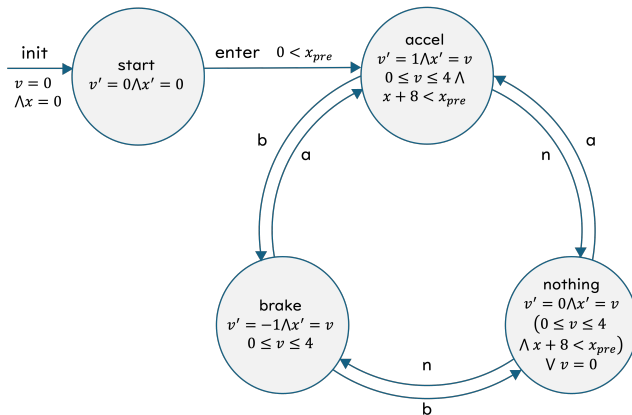
$$x(p_4) < x(p_3) < x(p_2) < x(p_1)$$

- x : position of a vehicle.
- v : velocity of a vehicle.
- loc : its pedal state (`start`, `accel`, `nothing`, or `brake`).
- `tick(T,S)`: advances time and updates position and velocity.
- If the distance becomes 8 units or less, the following vehicle must switch to `brake`.

Safety property to verify: Every following vehicle must not collide with its preceding vehicle.

5.1.1 Hybrid automaton of one vehicle

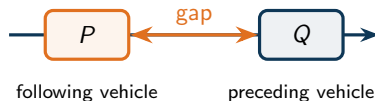
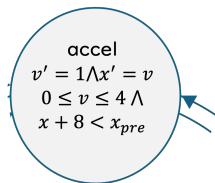
A location shows its label, flow constraint, and invariant condition. A transition shows its name and effective condition, if any.



The vehicle has the initial location **start** and three locations corresponding to pedal actions: **accel**, **brake**, and **nothing**.

5.1.2 Invariant condition at `accel`

At location `accel`, the invariant condition requires velocity to stay between 0 and 4 and the distance to the preceding vehicle to remain greater than 8.



- `tick(T,S)` can advance time only if the invariant condition is satisfied after the time advancement.
- If the safe distance cannot be maintained, the following vehicle must change to `brake`.

Invariant at `accel`: $x(P) + 8 < x(Q)$ **Safety property:** $x(P) < x(Q)$

5.1.3 Observation `ps` and `PSet`

Unlike the Fischer example, here `PSet` represents a sequence of processes. The observation `ps(S)` returns the sequence of processes that have entered the target lane.



$$x(p_4) < x(p_3) < x(p_2) < x(p_1)$$

- `ps(S) = p4 p3 p2 p1`
- `last(p4 p3 p2 p1) = p4`.
- `last(PS)` is used by the effective condition of `enter`.

```
[Pid < PSet]
```

```
Vehicle/main.cafe (excerpt)
```

```
op nil : -> PSet
```

```
op _ _ : PSet PSet -> PSet
```

```
{assoc idr: nil}
```

```
ops first last : PSet -> Pid
```

```
...
```

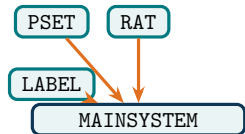
```
eq last(nil) = dummy .
```

```
eq last(P) = P .
```

```
eq last(P PS) = P .
```

5.1.4 System module of the vehicle model

The system module is named `MAINSYSTEM`. It imports `PSET`, `RAT`, and `LABEL` and declares the observations and transitions of the vehicle system.



- `ps`: sequence of vehicles.
- `x` and `v`: position and velocity.
- `loc`: pedal state.
- `now`: elapsed time.
- `tick`: time-advancing transition.

```
pr(PSET + RAT + LABEL)
*[Sys]*
op init : -> Sys
bop now : Sys -> Rat
bop ps : Sys -> PSet
bops v x : Pid Sys -> Rat
bop loc : Pid Sys -> Label
bops a b n enter : Pid Sys -> Sys
bop c-enter : Pid Sys -> Bool
...
bop tick : Rat Sys -> Sys
op c-tick : Rat Sys -> Bool
op c-tick : PSet Rat Sys -> Bool
```

Vehicle/main.cafe (excerpt)

5.1.5 Initial state

Initially every vehicle is at `start`; position, velocity, and elapsed time are 0, and `ps` is `nil`.

`init`

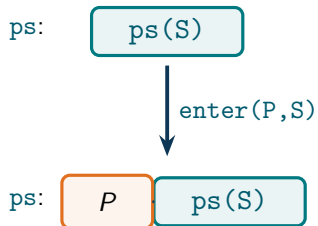
```
now: 0
ps: nil
loc(P): start
x(P): 0
v(P): 0
```

```
eq now(init) = 0 .
eq ps(init) = nil .
eq loc(P,init) = start .
eq v(P,init) = 0 .
eq x(P,init) = 0 .
```

Vehicle/main.cafe (excerpt)

5.1.6 Transition enter

Transition `enter` adds vehicle `P` to `ps` and changes its location from `start` to `accel`.

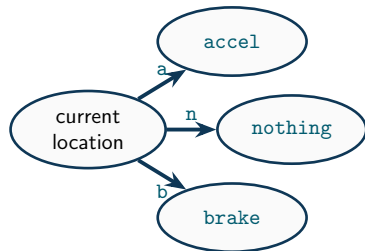


```
eq c-enter(P,S) =  
  (loc(P,S) = start) and  
  (ps(S) = nil or  
   x(last(ps(S)),S) > 0) .  
ceq ps(enter(P,S)) = P ps(S)  
  if c-enter(P,S) .  
ceq loc(P',enter(P,S)) =  
  (if P' = P then accel else loc(P',S) fi)  
  if c-enter(P,S) .
```

Vehicle/main.cafe (excerpt)

5.1.7 Discrete transitions for pedal actions

Transitions **a**, **b**, and **n** change **loc** to **accel**, **brake**, and **nothing**, respectively. Position **x**, velocity **v**, elapsed time **now**, and sequence **ps** are unchanged.



```
eq c-a(P,S) =  
  not (loc(P,S) = start) .  
ceq a(P,S) = S if not c-a(P,S) .  
eq now(a(P,S)) = now(S) .  
eq ps(a(P,S)) = ps(S) .  
ceq loc(P',a(P,S)) =  
  (if P' = P then accel  
   else loc(P',S) fi)  
  if c-a(P,S) .  
eq v(P',a(P,S)) = v(P',S) .  
eq x(P',a(P,S)) = x(P',S) .
```

Vehicle/main.cafe (excerpt)

Transitions **b** and **n** are defined in the same form.

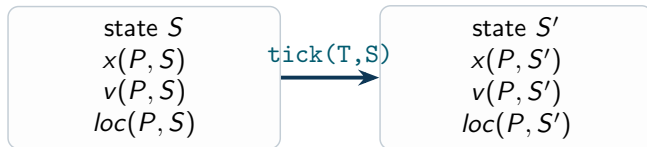
5.1.8 Continuous transition tick

`tick(T,S)` advances time by `T`. Position `x` and velocity `v` change according to the current location, while `loc` is unchanged.

`accel`
 $a = 1$

`nothing`
 $a = 0$

`brake`
 $a = -1$



$$v(P, S') = v(P, S) + aT$$

$$x(P, S') = x(P, S) + v(P, S)T + \frac{1}{2}aT^2$$

5.1.9 Flow equations in CafeOBJ

Operation `ac` gives acceleration 1, -1, or 0 for `accel`, `brake`, or `nothing`. Operations `nextv` and `nextx` compute velocity and position after `T` time units.

```
op ac : Label -> Rat
Vehicle/main.cafe (excerpt)
eq ac(accel) = 1 .
eq ac(brake) = -1 .
eq ac(nothing) = 0 .
eq nextv(P,T,S) =
  v(P,S) + ac(loc(P,S)) * T .
eq nextx(P,T,S) =
  v(P,S) * T
  + 1/2 * ac(loc(P,S)) * T * T
  + x(P,S) .
```

```
ceq v(P,tick(T,S)) =
  (if loc(P,S) = start then 0
   else nextv(P,T,S) fi)
  if c-tick(T,S) .
ceq x(P,tick(T,S)) =
  (if loc(P,S) = start then 0
   else nextx(P,T,S) fi)
  if c-tick(T,S) .
```

$$v(P, S') = v(P, S) + aT,$$

$$x(P, S') = x(P, S) + v(P, S)T + \frac{1}{2}aT^2.$$

5.1.10 Effective condition of tick for one and two vehicles

`c-tick(P,T,S)` checks the velocity of one vehicle. `c-tick(P Q,T,S)` also checks the distance between two consecutive vehicles.

```
eq c-tick(P,T,S) =  
  ((loc(P,S) = brake or loc(P,S) = accel or loc(P,S) = nothing)  
   implies (0 <= nextv(P,T,S) and nextv(P,T,S) <= 4)) .  
eq c-tick(P Q,T,S) =  
  c-tick(Q,T,S) and c-tick(P,T,S) and  
  (loc(P,S) = accel implies nextx(P,T,S) + 8 < nextx(Q,T,S)) and  
  (loc(P,S) = nothing implies  
   (((0 < nextv(P,T,S) and nextv(P,T,S) <= 4) and  
    nextx(P,T,S) + 8 < nextx(Q,T,S))  
    or nextv(P,T,S) = 0)) .
```

Vehicle/main.cafe (excerpt)

For one vehicle:

$$0 \leq \text{nextv}(P, T, S) \leq 4.$$

For consecutive vehicles P and Q :

In `accel`, their distance must remain greater than 8. In `nothing`, the same condition is required while velocity is positive; velocity 0 is also allowed.

5.1.11 Recursive c-tick over $ps(S)$

$c\text{-tick}(ps(S), T, S)$ recursively checks each vehicle and each consecutive pair in the finite sequence $ps(S)$.



$$x(p_4) < x(p_3) < x(p_2) < x(p_1)$$

Individual checks for p_1, p_2, p_3, p_4

Pair checks for $(p_2, p_1), (p_3, p_2), (p_4, p_3)$

Although the model allows an arbitrary number of vehicles, $ps(S)$ is finite in each reachable state.

```
eq c-tick(nil,T,S) = true
```

Vehicle/main.cafe (excerpt)

```
eq c-tick(P Q PS,T,S) =  
  c-tick(Q PS,T,S) and  
  c-tick(P Q,T,S) .
```

```
eq c-tick(T,S) =  
  (0 <= T) and c-tick(ps(S),T,S) .
```

5.2 Verification of the safety property

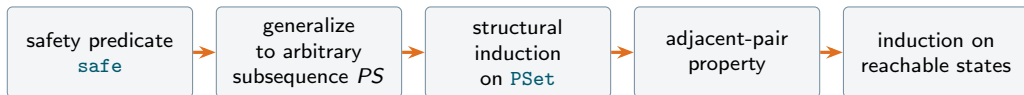
The safety property is that, in every reachable state, each following vehicle P remains behind its preceding vehicle Q :



$$x(p_4) < x(p_3) < x(p_2) < x(p_1)$$

$x(P, S) < x(Q, S)$ for each following vehicle P and its preceding vehicle Q .

Steps for verifying $x(P, S) < x(Q, S)$



5.2.1 Recursive predicate `safe`

Predicate `safe` recursively checks the position order of vehicles in a `PSet` sequence.

```
eq safe(nil,S) = true
eq safe(P,S) = true .
eq safe(P Q,S) =
  (x(P,S) < x(Q,S)) .
eq safe(P Q PS,S) =
  safe(Q PS,S) and
  safe(P Q,S) .
```

Vehicle/main.cafe (excerpt)



$$x(p_4) < x(p_3) < x(p_2) < x(p_1)$$

`safe(p4 p3 p2 p1,S)`

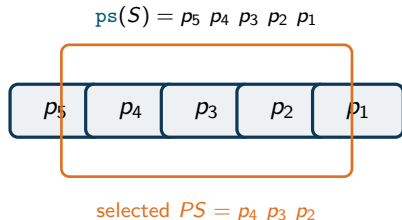


$$\begin{aligned} & x(p_4, S) < x(p_3, S) \\ & \wedge x(p_3, S) < x(p_2, S) \\ & \wedge x(p_2, S) < x(p_1, S) \end{aligned}$$

Verification goal: for every reachable state S , `safe(ps(S),S)` holds.

5.2.2 Generalizing the safety property

To use structural induction, we generalize the goal to an arbitrary subsequence PS of $\text{ps}(S)$.

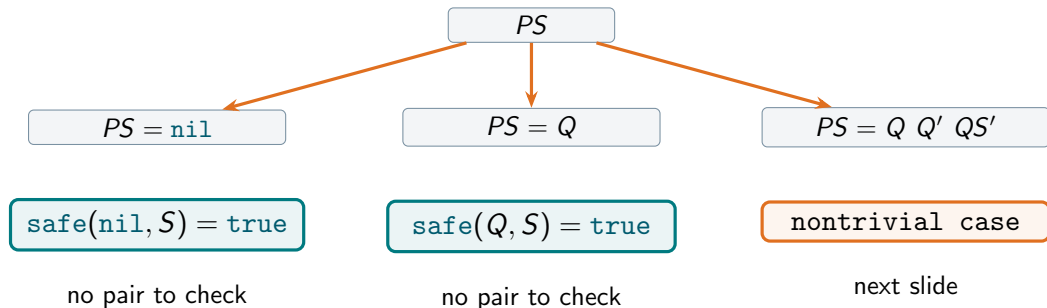


$\forall S$ reachable from `init`.
 $\forall PS \in \text{PSet}$.
 $PS \text{ sub } \text{ps}(S) \Rightarrow \text{safe}(PS, S)$

- The operator `sub` denotes a subsequence.
- Since $PS \text{ sub } PS$ holds, taking $PS = \text{ps}(S)$ gives the original safety property.
- Therefore the generalized property is stronger.

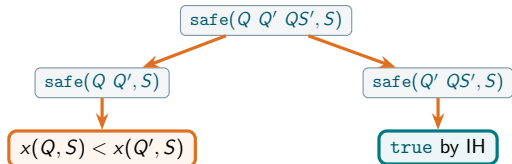
5.2.3 Structural induction on PSet

We consider the following three cases for PS .



5.2.4 Nontrivial constructor case

Prove the case of $PS = Q \ Q' \ QS'$.



$$\begin{aligned} \text{safe}(Q \ Q' \ QS', S) &= \text{safe}(Q \ Q', S) \\ &\quad \wedge \text{safe}(Q' \ QS', S), \\ \text{safe}(Q \ Q', S) &= x(Q, S) < x(Q', S). \end{aligned}$$

- The induction hypothesis proves the tail $\text{safe}(Q' \ QS', S)$.
- The remaining goal is the adjacent-pair inequality.

This remaining property is proved over reachable states by the Proof Score method.

Complete proof: [Vehicle/safety.cafe](#)

5.2.5 Adjacent-pair invariant over reachable states

We prove the remaining position property for every reachable state with invariant `inv1`.

```
eq inv1(Q,Q',QS',S) =  
  (Q Q' QS' sub ps(S)  
   implies  
    (x(Q,S) < x(Q',S))) .
```

Vehicle/main.cafe (excerpt)

- $Q \ Q' \ QS' \text{ sub } ps(S)$ states that this sequence is a subsequence of $ps(S)$.
- `inv1` requires $x(Q, S) < x(Q', S)$.

Proof Score: `Vehicle/inv1.cafe`



following preceding

$$x(Q, S) < x(Q', S)$$

5.2.6 Completion of the vehicle proof

As in the previous examples, case-splitting and lemmata complete the Proof Score.

Lemmata: 5

Proof passages: 253

All reductions: `true`

Invariant Proof Scores: `Vehicle/inv1.cafe` ... `Vehicle/inv6.cafe`

Unlike the previous proof constructions, this case also required the following auxiliary lemma on `PSet`.

```
eq lemma1(QS,T,S) =  
  ((c-tick(T,S) and  
    (QS sub ps(S)))  
   implies c-tick(QS,T,S)) .
```

`Vehicle/main.cafe` (excerpt)

This lemma is proved by structural induction on the sequence `QS`.

Proof: `Vehicle/auxiliary/lemma1.cafe`

Outline

1. Introduction
2. The OTS/CafeOBJ Method Overview
3. Modeling Multitask Real-Time Systems in OTS/CafeOBJ Specifications
4. Case study: Fischer's protocol
5. Case study: autonomous vehicle control system
- 6. Conclusion**

6.1 Before proving: execute the specification

Before starting a proof, it is important to execute the specification and check that it models the intended behavior.

- 1 Construct representative execution traces.
- 2 Reduce observations and effective conditions.
- 3 Check that the expected behavior can actually occur.
- 4 Then prove the safety property.

Example execution fragment

```
eq s1 = ab(init,p1) .  
eq s2 = tick(2,s1) .  
eq s3 = bc(s2,p1) .  
red loc(s3,p1) .  
red c-tick(2,s1) .
```

A safety proof alone is not enough. For example, a specification in which no process can ever reach `cs` still satisfies the property that at most one process is in `cs`.

Execution helps us check that the specification also exhibits the intended behavior.

6.2 Automation around Proof Score construction

Three complementary research directions support interactive Proof Score construction.

Proof-structure automation

Case analysis and parts of Proof Score development can be automated.

Gaina et al. [11]

Specification translation + model checking

Translate a specification and use model checking to reject false conjectures on finite instances.

The general proof is still given by the Proof Score.

Nakamura et al. [12];
Zhang et al. [13]

Lemma conjecture support

r-SMGA and Maude help users formulate and test candidate characteristics.

Bui et al. [14]

6.3 LLM-assisted interactive Proof Score construction

An LLM can take over part of the human-computer interaction in interactive theorem proving such as the OTS/CafeOBJ method.

In our construction, it also replaced some of the support work described on the previous slide.

- repeated CafeOBJ execution and case-splitting;
- inspecting intermediate terms and false passages;
- suggesting additional invariant and lemma candidates for the Vehicle case study;
- checking simple arithmetic facts with Z3, an SMT solver [15], for example
$$8 + X < Z \implies X < Z.$$

The original papers treated such simple arithmetic facts as valid without explicit proofs. In this LLM-assisted construction, Z3 was used to check them explicitly.

Outline

1. Introduction
2. The OTS/CafeOBJ Method Overview
3. Modeling Multitask Real-Time Systems in OTS/CafeOBJ Specifications
4. Case study: Fischer's protocol
5. Case study: autonomous vehicle control system
6. Conclusion

The remaining slides contain References and Proof-score overviews.

- [1] R. Diaconescu and K. Futatsugi, *CafeOBJ Report*, World Scientific, 1998.
- [2] A. Riesco, K. Ogata, M. Nakamura, D. Gaina, D.D. Tran, and K. Futatsugi, "Proof Scores: A Survey," *ACM Comput. Surv.*, 57(10), Article 251, 2025. <https://doi.org/10.1145/3729166>
- [3] K. Ogata and K. Futatsugi, "Proof scores in the OTS/CafeOBJ method," *FMOODS 2003*, LNCS 2884, pp.170–184, 2003.
- [4] K. Ogata and K. Futatsugi, "Some tips on writing proof scores in the OTS/CafeOBJ method," *Goguen Festschrift*, LNCS 4060, pp.596–615, 2006.
- [5] K. Ogata and K. Futatsugi, "Modeling and verification of real-time systems based on equations," *Sci. Comput. Program.*, 66(2), pp.162–180, 2007.
- [6] M. Nakamura, S. Higashi, K. Sakakibara, and K. Ogata, "Specification and Verification of Multitask Real-Time Systems Using the OTS/CafeOBJ Method," *IEICE Trans. Fundamentals*, E105-A(5), pp.823–832, 2022. <https://doi.org/10.1587/transfun.2021MAP0007>
- [7] Y. Wang, M. Nakamura, R. Takano, T. Matsumoto, and K. Sakakibara, "Multitask Hybrid Observational Transition System and its Application," *IEICE Trans. Fundamentals*, E109-A(3), pp.638–650, 2026. <https://doi.org/10.1587/transfun.2025EAP1049>
- [8] M. Fischer, "Re: where are you?" Electronic mail message to L. Lamport, June 25, 1985.

- [9] L. Lamport, "A fast mutual exclusion algorithm," *ACM Trans. Comput. Syst.*, 5(1), pp.1–11, 1987.
- [10] E.M. Clarke, T.A. Henzinger, H. Veith, and R. Bloem (eds.), *Handbook of Model Checking*, Springer, 2018.
- [11] D. Gaina, D. Lucanu, K. Ogata, and K. Futatsugi, "On Automation of OTS/CafeOBJ Method," *Specification, Algebra, and Software*, LNCS 8373, pp.578–602, 2014. https://doi.org/10.1007/978-3-642-54624-2_29
- [12] M. Nakamura, W. Kong, K. Ogata, and K. Futatsugi, "A Specification Translation from Behavioral Specifications to Rewrite Specifications," *IEICE Trans. Inf. & Syst.*, E91-D(5), pp.1492–1503, 2008. <https://doi.org/10.1093/ietisy/e91-d.5.1492>
- [13] M. Zhang, K. Ogata, and M. Nakamura, "Translation of State Machines from Equational Theories into Rewrite Theories with Tool Support," *IEICE Trans. Inf. & Syst.*, E94-D(5), pp.976–988, 2011. <https://doi.org/10.1587/transinf.E94.D.976>
- [14] D.D. Bui, D.D. Tran, K. Ogata, and A. Riesco, "Integration of state machine graphical animation and Maude to facilitate characteristic conjecture: an approach to lemma discovery in theorem proving," *Multimed. Tools Appl.*, 83(12), pp.36865–36898, 2024. <https://doi.org/10.1007/s11042-023-15780-5>
- [15] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," *TACAS 2008*, LNCS 4963, pp.337–340, 2008. https://doi.org/10.1007/978-3-540-78800-3_24

A. Proof-score overviews

These compact maps summarize the accompanying Proof Scores used in this tutorial.

$$C_1 \wedge \neg C_2 \wedge C_3$$

$$\neg C_1$$

inv2



- Each row is one successful proof passage.
- The conjunction is its root-to-leaf case path.
- $C_i / \neg C_i$ are complementary cases when the accompanying source has that split.
- An empty terminal box uses no invariant lemma.
- A named terminal box records the lemma actually used by that passage.

When an effective-condition proposition abbreviates conditions shown separately in positive branches, its negated leaf also shows the equivalent expanded complement in parentheses.

Explicit ABCf split trees and exact exhaustive covers are represented without inventing parent-child relationships. Intermediate failed attempts are not shown.

A.1.1 ABCf — inv1: all transitions

transition **ab**

8 final passages split tree

$\neg C_1$	☐
$C_1 \wedge C_2$	☐
$C_1 \wedge \neg C_2 \wedge C_3 \wedge C_4$	☐
$C_1 \wedge \neg C_2 \wedge C_3 \wedge \neg C_4 \wedge C_5$	inv2
$C_1 \wedge \neg C_2 \wedge C_3 \wedge \neg C_4 \wedge \neg C_5$	☐
$C_1 \wedge \neg C_2 \wedge \neg C_3 \wedge C_4 \wedge C_6$	inv2
$C_1 \wedge \neg C_2 \wedge \neg C_3 \wedge C_4 \wedge \neg C_6$	☐
$C_1 \wedge \neg C_2 \wedge \neg C_3 \wedge \neg C_4$	☐

transition **bc**

4 final passages split tree

$\neg C_7$	☐
$C_7 \wedge C_8$	☐
$C_7 \wedge \neg C_8 \wedge C_9$	☐
$C_7 \wedge \neg C_8 \wedge \neg C_9$	☐

transition **init**

1 final passages split tree

$\top$	☐
--------	--------------------------

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.1.1 ABCf — inv1: case-proposition key

C_1 $c\text{-}ab(s, p3)$

C_3 $p2 = p3$

C_5 $loc(s, p1) = b$

C_7 $c\text{-}bc(s, p3)$

C_9 $p3 = p2$

C_2 $p1 = p2$

C_4 $p1 = p3$

C_6 $loc(s, p2) = b$

C_8 $p3 = p1$

A.1.2 ABCf — inv2: all transitions

transition **bc**

5 final passages split tree

$\neg C_2$ ☐

$C_2 \wedge C_3$ ☐

$C_2 \wedge \neg C_3 \wedge C_4 \wedge C_5$ **inv1**

$C_2 \wedge \neg C_3 \wedge C_4 \wedge \neg C_5$ ☐

$C_2 \wedge \neg C_3 \wedge \neg C_4$ ☐

transition **ab**

2 final passages split tree

$\neg C_1$ ☐

C_1 ☐

transition **init**

1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.1.2 ABCf — inv2: case-proposition key

C_1 $c\text{-}ab(s,p3)$

C_3 $p1 = p3$

C_5 $flag(s)$

C_2 $c\text{-}bc(s,p3)$

C_4 $loc(s,p1) = b$

A.2.1 Fischer — inv1: transitions (1/4)

transition check

8 final passages exact cover

$C_1 \wedge C_2 \wedge C_3 \wedge C_4$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge C_5$	inv2
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4 \wedge C_5$	inv2
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐
$\neg C_7 (= \neg(C_1 \wedge C_2))$	☐

transition exit

6 final passages exact cover

$C_8 \wedge C_3 \wedge C_4$	☐
$C_8 \wedge C_3 \wedge \neg C_4$	☐
$C_8 \wedge \neg C_3 \wedge C_4$	☐
$C_8 \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐
$C_8 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐
$\neg C_9 (= \neg C_8)$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.1 Fischer — inv1: transitions (2/4)

transition **reset** 6 final passages exact cover

$C_{10} \wedge C_3 \wedge C_4$ ☐

$C_{10} \wedge \neg C_3 \wedge C_4$ ☐

$C_{10} \wedge C_3 \wedge \neg C_4$ ☐

$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{11} (= \neg C_{10})$ ☐

transition **init** 1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.1 Fischer — inv1: transitions (3/4)

transition set **6 final passages** exact cover

$C_{12} \wedge C_3 \wedge C_4$ ☐

$C_{12} \wedge C_3 \wedge \neg C_4$ ☐

$C_{12} \wedge \neg C_3 \wedge C_4$ ☐

$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{13} (= \neg C_{12})$ ☐

transition test **6 final passages** exact cover

$C_{14} \wedge C_{15} \wedge C_3 \wedge C_4$ ☐

$C_{14} \wedge C_{15} \wedge C_3 \wedge \neg C_4$ ☐

$C_{14} \wedge C_{15} \wedge \neg C_3 \wedge C_4$ ☐

$C_{14} \wedge C_{15} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{14} \wedge C_{15} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{16} (= \neg(C_{14} \wedge C_{15}))$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.1 Fischer — inv1: transitions (4/4)

transition **try** 6 final passages exact cover

$C_{20} \wedge C_3 \wedge C_4$ ☐

$C_{20} \wedge C_3 \wedge \neg C_4$ ☐

$C_{20} \wedge \neg C_3 \wedge C_4$ ☐

$C_{20} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{20} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{21} (= \neg C_{20})$ ☐

transition **tick** 2 final passages exact cover

$C_{17} \wedge C_{18}$ ☐

$\neg C_{19} (= \neg(C_{17} \wedge C_{18}))$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.1 Fischer — inv1: case-proposition key

C_1 $\text{loc}(s,k) = d$

C_4 $j = k$

C_7 $\text{c-check}(s,k)$

C_{10} $\text{loc}(s,k) = e$

C_{13} $\text{c-set}(s,k)$

C_{16} $\text{c-test}(s,k)$

C_{19} $\text{c-tick}(t,s)$

C_2 $\text{l}(s,k) \leq \text{now}(s)$

C_5 $\text{turn}(s) = k$

C_8 $\text{loc}(s,k) = \text{cs}$

C_{11} $\text{c-reset}(s,k)$

C_{14} $\text{loc}(s,k) = b$

C_{17} $\text{now}(s) + t \leq \min(u(s))$

C_{20} $\text{loc}(s,k) = a$

C_3 $i = k$

C_6 $i = j$

C_9 $\text{c-exit}(s,k)$

C_{12} $\text{loc}(s,k) = c$

C_{15} $\text{turn}(s) = \text{nop}$

C_{18} $0 \leq t$

C_{21} $\text{c-try}(s,k)$

A.2.2 Fischer — inv2: transitions (1/4)

transition **check**

9 final passages exact cover

$C_1 \wedge C_2 \wedge C_3 \wedge C_4$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge C_5 \wedge \neg C_6$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge C_5 \wedge C_6$	☐ inv3, inv5
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4 \wedge C_7$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4 \wedge \neg C_7$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge C_8$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8$	☐
$\neg C_9 (= \neg(C_1 \wedge C_2))$	☐

transition **exit**

6 final passages exact cover

$C_{10} \wedge C_3 \wedge C_4$	☐
$C_{10} \wedge C_3 \wedge \neg C_4$	☐
$C_{10} \wedge \neg C_3 \wedge C_4$	☐
$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8$	☐
$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8$	☐
$\neg C_{11} (= \neg C_{10})$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.2 Fischer — inv2: transitions (2/4)

transition **reset**

11 final passages exact cover

$C_{12} \wedge C_3 \wedge C_4$	☐
$C_{12} \wedge C_3 \wedge \neg C_4$	☐
$C_{12} \wedge \neg C_3 \wedge C_4 \wedge C_7$	☐
$C_{12} \wedge \neg C_3 \wedge C_4 \wedge \neg C_7$	☐
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8$	☐
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8 \wedge C_7 \wedge C_{13} \wedge C_6$	☐
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8 \wedge C_7 \wedge C_{13} \wedge \neg C_6$	☐
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8 \wedge C_7 \wedge \neg C_{13} \wedge C_{14}$	☐
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8 \wedge C_7 \wedge \neg C_{13} \wedge \neg C_{14}$	☐
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8 \wedge \neg C_7$	☐
$\neg C_{15} (= \neg C_{12})$	☐

transition **init**

1 final passage split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.2 Fischer — inv2: transitions (3/4)

transition set

8 final passages exact cover

$C_{16} \wedge C_3 \wedge C_4$	☐
$C_{16} \wedge C_3 \wedge \neg C_4$	☐
$C_{16} \wedge \neg C_3 \wedge C_4$	☐
$C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8 \wedge C_{17} \wedge C_{18}$	☐
$C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8 \wedge C_{17} \wedge \neg C_{18}$	☐
$C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8 \wedge \neg C_{17}$	☐
$C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8$	☐
$\neg C_{19} (= \neg C_{16})$	☐

transition test

6 final passages exact cover

$C_{20} \wedge C_{21} \wedge C_3 \wedge C_4$	☐
$C_{20} \wedge C_{21} \wedge C_3 \wedge \neg C_4$	☐
$C_{20} \wedge C_{21} \wedge \neg C_3 \wedge C_4$	☐
$C_{20} \wedge C_{21} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8$	☐
$C_{20} \wedge C_{21} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8$	☐
$\neg C_{22} (= \neg(C_{20} \wedge C_{21}))$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.2 Fischer — inv2: transitions (4/4)

transition **try** 6 final passages exact cover

$$C_{26} \wedge C_3 \wedge C_4 \quad \square$$

$$C_{26} \wedge C_3 \wedge \neg C_4 \quad \square$$

$$C_{26} \wedge \neg C_3 \wedge C_4 \quad \square$$

$$C_{26} \wedge \neg C_3 \wedge \neg C_4 \wedge C_8 \quad \square$$

$$C_{26} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_8 \quad \square$$

$$\neg C_{27} (= \neg C_{26}) \quad \square$$

transition **tick** 2 final passages exact cover

$$C_{23} \wedge C_{24} \quad \square$$

$$\neg C_{25} (= \neg(C_{23} \wedge C_{24})) \quad \square$$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.2 Fischer — inv2: case-proposition key

C_1 `loc(s,k) = d`

C_5 `turn(s) = k`

C_9 `c-check(s,k)`

C_{13} `loc(s,i) = cs`

C_{17} `turn(s) = j`

C_{21} `turn(s) = nop`

C_{25} `c-tick(t,s)`

C_2 `l(s,k) <= now(s)`

C_6 `loc(s,j) = c`

C_{10} `loc(s,k) = cs`

C_{14} `loc(s,i) = e`

C_{18} `loc(s,j) = cs`

C_{22} `c-test(s,k)`

C_{26} `loc(s,k) = a`

C_3 `i = k`

C_7 `turn(s) = i`

C_{11} `c-exit(s,k)`

C_{15} `c-reset(s,k)`

C_{19} `c-set(s,k)`

C_{23} `now(s) + t <= min(u(s))`

C_{27} `c-try(s,k)`

C_4 `j = k`

C_8 `i = j`

C_{12} `loc(s,k) = e`

C_{16} `loc(s,k) = c`

C_{20} `loc(s,k) = b`

C_{24} `0 <= t`

A.2.3 Fischer — inv3: transitions (1/4)

transition **check**

8 final passages exact cover

$C_1 \wedge C_2 \wedge C_3 \wedge C_4$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4 \wedge C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge C_5$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐
$\neg C_7 (= \neg(C_1 \wedge C_2))$	☐

transition **exit**

6 final passages exact cover

$C_8 \wedge C_3 \wedge C_4$	☐
$C_8 \wedge \neg C_3 \wedge C_4$	☐
$C_8 \wedge C_3 \wedge \neg C_4$	☐
$C_8 \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐
$C_8 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐
$\neg C_9 (= \neg C_8)$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.3 Fischer — inv3: transitions (2/4)

transition **reset** 6 final passages exact cover

$C_{10} \wedge C_3 \wedge C_4$ ☐

$C_{10} \wedge \neg C_3 \wedge C_4$ ☐

$C_{10} \wedge C_3 \wedge \neg C_4$ ☐

$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{11} (= \neg C_{10})$ ☐

transition **init** 1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.3 Fischer — inv3: transitions (3/4)

transition set

7 final passages exact cover

$C_{12} \wedge C_3 \wedge C_4$	☐	
$C_{12} \wedge \neg C_3 \wedge C_4$	☐	
$C_{12} \wedge C_3 \wedge \neg C_4 \wedge C_{13}$	☐	inv4, wf-u
$C_{12} \wedge C_3 \wedge \neg C_4 \wedge \neg C_{13}$	☐	
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐	
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐	
$\neg C_{14} (= \neg C_{12})$	☐	

transition test

6 final passages exact cover

$C_{15} \wedge C_{16} \wedge C_3 \wedge C_4$	☐	
$C_{15} \wedge C_{16} \wedge \neg C_3 \wedge C_4$	☐	
$C_{15} \wedge C_{16} \wedge C_3 \wedge \neg C_4$	☐	
$C_{15} \wedge C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐	
$C_{15} \wedge C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐	
$\neg C_{17} (= \neg(C_{15} \wedge C_{16}))$	☐	

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.3 Fischer — inv3: transitions (4/4)

transition **tick** 6 final passages exact cover

$$C_{18} \wedge C_{19} \wedge C_{20} \wedge C_{21} \wedge C_{13} \wedge C_6 \quad \square$$

$$C_{18} \wedge C_{19} \wedge C_{20} \wedge C_{21} \wedge C_{13} \wedge \neg C_6 \quad \square$$

$$C_{18} \wedge C_{19} \wedge C_{20} \wedge C_{21} \wedge \neg C_{13} \quad \square$$

$$C_{18} \wedge C_{19} \wedge C_{20} \wedge \neg C_{21} \quad \square$$

$$C_{18} \wedge C_{19} \wedge \neg C_{20} \quad \square$$

$$\neg C_{22} (= \neg(C_{18} \wedge C_{19})) \quad \square$$

transition **try** 6 final passages exact cover

$$C_{23} \wedge C_3 \wedge C_4 \quad \square$$

$$C_{23} \wedge \neg C_3 \wedge C_4 \quad \square$$

$$C_{23} \wedge C_3 \wedge \neg C_4 \quad \square$$

$$C_{23} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6 \quad \square$$

$$C_{23} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6 \quad \square$$

$$\neg C_{24} (= \neg C_{23}) \quad \square$$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.3 Fischer — inv3: case-proposition key

C_1 $\text{loc}(s,k) = d$

C_4 $j = k$

C_7 $\text{c-check}(s,k)$

C_{10} $\text{loc}(s,k) = e$

C_{13} $\text{loc}(s,j) = c$

C_{16} $\text{turn}(s) = \text{nop}$

C_{19} $0 \leq t$

C_{22} $\text{c-tick}(t,s)$

C_2 $\text{l}(s,k) \leq \text{now}(s)$

C_5 $\text{turn}(s) = k$

C_8 $\text{loc}(s,k) = \text{cs}$

C_{11} $\text{c-reset}(s,k)$

C_{14} $\text{c-set}(s,k)$

C_{17} $\text{c-test}(s,k)$

C_{20} $\text{loc}(s,i) = d$

C_{23} $\text{loc}(s,k) = a$

C_3 $i = k$

C_6 $i = j$

C_9 $\text{c-exit}(s,k)$

C_{12} $\text{loc}(s,k) = c$

C_{15} $\text{loc}(s,k) = b$

C_{18} $\text{now}(s) + t \leq \text{min}(u(s))$

C_{21} $\text{turn}(s) = i$

C_{24} $\text{c-try}(s,k)$

A.2.4 Fischer — inv4: transitions (1/4)

transition **check**

8 final passages exact cover

$C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge C_4$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge C_5$	☐
$C_1 \wedge C_2 \wedge C_3 \wedge \neg C_4 \wedge \neg C_5$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐
$C_1 \wedge C_2 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐
$\neg C_7 (= \neg(C_1 \wedge C_2))$	☐

transition **exit**

6 final passages exact cover

$C_8 \wedge C_3 \wedge C_4$	☐
$C_8 \wedge \neg C_3 \wedge C_4$	☐
$C_8 \wedge C_3 \wedge \neg C_4$	☐
$C_8 \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐
$C_8 \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐
$\neg C_9 (= \neg C_8)$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.4 Fischer — inv4: transitions (2/4)

transition **reset** 6 final passages exact cover

$C_{10} \wedge C_3 \wedge C_4$ ☐

$C_{10} \wedge \neg C_3 \wedge C_4$ ☐

$C_{10} \wedge C_3 \wedge \neg C_4$ ☐

$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{10} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{11} (= \neg C_{10})$ ☐

transition **init** 1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.4 Fischer — inv4: transitions (3/4)

transition set

7 final passages exact cover

$C_{12} \wedge C_3 \wedge C_4$	☐	
$C_{12} \wedge \neg C_3 \wedge C_4 \wedge C_{13}$	☐	wf-u
$C_{12} \wedge \neg C_3 \wedge C_4 \wedge \neg C_{13}$	☐	
$C_{12} \wedge C_3 \wedge \neg C_4$	☐	
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐	wf-u
$C_{12} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐	wf-u
$\neg C_{14} (= \neg C_{12})$	☐	

transition test

6 final passages exact cover

$C_{15} \wedge C_{16} \wedge C_3 \wedge C_4$	☐	
$C_{15} \wedge C_{16} \wedge \neg C_3 \wedge C_4$	☐	
$C_{15} \wedge C_{16} \wedge C_3 \wedge \neg C_4$	☐	
$C_{15} \wedge C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$	☐	
$C_{15} \wedge C_{16} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$	☐	
$\neg C_{17} (= \neg(C_{15} \wedge C_{16}))$	☐	

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.4 Fischer — inv4: transitions (4/4)

transition **try** 6 final passages exact cover

$C_{22} \wedge C_3 \wedge C_4$ ☐

$C_{22} \wedge \neg C_3 \wedge C_4$ ☐

$C_{22} \wedge C_3 \wedge \neg C_4$ ☐

$C_{22} \wedge \neg C_3 \wedge \neg C_4 \wedge C_6$ ☐

$C_{22} \wedge \neg C_3 \wedge \neg C_4 \wedge \neg C_6$ ☐

$\neg C_{23} (= \neg C_{22})$ ☐

transition **tick** 4 final passages exact cover

$C_{18} \wedge C_{19} \wedge C_{13} \wedge C_{20}$ ☐

$C_{18} \wedge C_{19} \wedge C_{13} \wedge \neg C_{20}$ ☐

$C_{18} \wedge C_{19} \wedge \neg C_{13}$ ☐

$\neg C_{21} (= \neg(C_{18} \wedge C_{19}))$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.4 Fischer — inv4: case-proposition key

C_1 $\text{loc}(s,k) = d$

C_4 $j = k$

C_7 $\text{c-check}(s,k)$

C_{10} $\text{loc}(s,k) = e$

C_{13} $\text{loc}(s,i) = c$

C_{16} $\text{turn}(s) = \text{nop}$

C_{19} $0 \leq d$

C_{22} $\text{loc}(s,k) = a$

C_2 $l(s,k) \leq \text{now}(s)$

C_5 $\text{turn}(s) = k$

C_8 $\text{loc}(s,k) = \text{cs}$

C_{11} $\text{c-reset}(s,k)$

C_{14} $\text{c-set}(s,k)$

C_{17} $\text{c-test}(s,k)$

C_{20} $\text{get}(i,u(s)) < (\text{now}(s) +$
 $d_2)$

C_{23} $\text{c-try}(s,k)$

C_3 $i = k$

C_6 $i = j$

C_9 $\text{c-exit}(s,k)$

C_{12} $\text{loc}(s,k) = c$

C_{15} $\text{loc}(s,k) = b$

C_{18} $\text{now}(s) + d \leq \min(u(s))$

C_{21} $\text{c-tick}(d,s)$

A.2.5 Fischer — inv5: transitions (1/4)

transition **check** 2 final passages exact cover

$C_1 \wedge C_2$

$\neg C_3 (= \neg(C_1 \wedge C_2))$

transition **exit** 2 final passages exact cover

C_4

$\neg C_5 (= \neg C_4)$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.5 Fischer — inv5: transitions (2/4)

transition `reset` **2 final passages** exact cover

C_6

$\neg C_7 (= \neg C_6)$

transition `init` **1 final passages** split tree

$\top$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.5 Fischer — inv5: transitions (3/4)

transition set 6 final passages exact cover

$C_8 \wedge C_9 \wedge C_{10}$	wf-u
$C_8 \wedge \neg C_9 \wedge C_{10}$	wf-u
$C_8 \wedge C_9 \wedge \neg C_{10}$	wf-u
$C_8 \wedge \neg C_9 \wedge \neg C_{10} \wedge C_{11}$	wf-u
$C_8 \wedge \neg C_9 \wedge \neg C_{10} \wedge \neg C_{11}$	wf-u
$\neg C_{12} (= \neg C_8)$	

transition test 6 final passages exact cover

$C_{13} \wedge C_{14} \wedge C_9 \wedge C_{10}$	
$C_{13} \wedge C_{14} \wedge \neg C_9 \wedge C_{10}$	
$C_{13} \wedge C_{14} \wedge C_9 \wedge \neg C_{10}$	
$C_{13} \wedge C_{14} \wedge \neg C_9 \wedge \neg C_{10} \wedge C_{11}$	
$C_{13} \wedge C_{14} \wedge \neg C_9 \wedge \neg C_{10} \wedge \neg C_{11}$	
$\neg C_{15} (= \neg(C_{13} \wedge C_{14}))$	

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.5 Fischer — inv5: transitions (4/4)

transition **tick** 2 final passages exact cover

$C_{16} \wedge C_{17}$ wf-u

$\neg C_{18} (= \neg(C_{16} \wedge C_{17}))$

transition **try** 2 final passages exact cover

C_{19}

$\neg C_{20} (= \neg C_{19})$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.2.5 Fischer — inv5: case-proposition key

C_1 $\text{loc}(s,k) = d$

C_4 $\text{loc}(s,k) = \text{cs}$

C_7 $\text{c-reset}(s,k)$

C_{10} $j = k$

C_{13} $\text{loc}(s,k) = b$

C_{16} $\text{now}(s) + d \leq \min(u(s))$

C_{19} $\text{loc}(s,k) = a$

C_2 $\text{l}(s,k) \leq \text{now}(s)$

C_5 $\text{c-exit}(s,k)$

C_8 $\text{loc}(s,k) = c$

C_{11} $i = j$

C_{14} $\text{turn}(s) = \text{nop}$

C_{17} $0 \leq d$

C_{20} $\text{c-try}(s,k)$

C_3 $\text{c-check}(s,k)$

C_6 $\text{loc}(s,k) = e$

C_9 $i = k$

C_{12} $\text{c-set}(s,k)$

C_{15} $\text{c-test}(s,k)$

C_{18} $\text{c-tick}(d,s)$

A.3.1 Autonomous vehicle — inv1: non-tick transitions (1/2)

transition enter 8 final passages exact cover (overlap permitted)

$$C_3 \wedge C_4 \wedge C_5 \wedge C_6 \wedge C_7 \wedge C_8 \quad \square$$

$$C_3 \wedge C_4 \wedge C_5 \wedge C_6 \wedge C_7 \wedge \neg C_8 \wedge C_9 \quad \square$$

$$C_3 \wedge C_4 \wedge C_5 \wedge C_6 \wedge C_7 \wedge \neg C_8 \wedge \neg C_9 \quad \text{inv2, inv5}$$

$$C_3 \wedge C_4 \wedge C_5 \wedge C_6 \wedge \neg C_7 \quad \square$$

$$C_3 \wedge C_4 \wedge C_5 \wedge \neg C_6 \quad \square$$

$$C_3 \wedge C_4 \wedge \neg C_5 \quad \square$$

$$C_{10} \wedge C_{11} \quad \square$$

$$\neg C_{10} (= \neg(C_3 \wedge (C_4 \vee C_{11}))) \quad \square$$

transition a

2 final passages exact cover

$$C_1 \quad \square$$

$$\neg C_1 \quad \square$$

transition b

2 final passages exact cover

$$C_2 \quad \square$$

$$\neg C_2 \quad \square$$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.1 Autonomous vehicle — inv1: non-tick transitions (2/2)

transition **n**

2 final passages exact cover



transition **init**

1 final passages split tree



Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.1 Autonomous vehicle — inv1: transition tick (1/4)

transition tick (1/7) 10 final passages exact cover (overlap permitted)

$C_{13} \wedge C_{14} \wedge C_{15} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge C_{14} \wedge C_{15} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge C_{14} \wedge C_{15} \wedge \neg C_9$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{17} \wedge C_{18}$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{18} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{18} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{18} \wedge \neg C_9$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{19} \wedge C_{20}$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{20} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{20} \wedge C_9 \wedge \neg C_{16}$	

transition tick (2/7) 10 final passages exact cover (overlap permitted)

$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{20} \wedge \neg C_9$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{22}$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{22} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{22} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{22} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{15} \wedge C_{24}$	
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{15} \wedge \neg C_{24} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{15} \wedge \neg C_{24} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{15} \wedge \neg C_{24} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{17} \wedge C_{25}$	

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.1 Autonomous vehicle — inv1: transition tick (2/4)

transition tick (3/7) 10 final passages exact cover (overlap permitted)

$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{17} \wedge C_9 \wedge C_{16}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{25} \wedge C_9 \wedge \neg C_{16}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{25} \wedge \neg C_9$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{19} \wedge C_{26}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{19} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{26} \wedge C_9 \wedge \neg C_{16}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{26} \wedge \neg C_9$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{21} \wedge C_{27}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{21} \wedge C_{16}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{21} \wedge \neg C_{27} \wedge C_9 \wedge \neg C_{16}$	☐

transition tick (4/7) 10 final passages exact cover (overlap permitted)

$C_{13} \wedge \neg C_{14} \wedge C_{23} \wedge C_{21} \wedge \neg C_{27} \wedge \neg C_9$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge C_{15} \wedge C_{29}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge C_{15} \wedge \neg C_{29} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge C_{15} \wedge \neg C_{29} \wedge C_9 \wedge \neg C_{16}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge C_{15} \wedge \neg C_{29} \wedge \neg C_9$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{17} \wedge C_{30}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{17} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{30} \wedge C_9 \wedge \neg C_{16}$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{30} \wedge \neg C_9$	☐
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{19} \wedge C_{31}$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.1 Autonomous vehicle — inv1: transition tick (3/4)

transition tick (5/7) 10 final passages exact cover (overlap permitted)

$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{19} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{31} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{31} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{21} \wedge C_{32}$	
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{21} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{32} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge \neg C_{14} \wedge C_{28} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{32} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge C_{15} \wedge C_{34}$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge C_{15} \wedge \neg C_{34} \wedge C_9 \wedge C_{16}$	inv3, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge C_{15} \wedge \neg C_{34} \wedge C_9 \wedge \neg C_{16}$	

transition tick (6/7) 10 final passages exact cover (overlap permitted)

$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge C_{15} \wedge \neg C_{34} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{17} \wedge C_{35}$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{35} \wedge C_9 \wedge C_{16}$	inv4, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{35} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{17} \wedge \neg C_{35} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{19} \wedge C_{36}$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{36} \wedge C_9 \wedge C_{16}$	inv3, inv4, inv6
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{36} \wedge C_9 \wedge \neg C_{16}$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{19} \wedge \neg C_{36} \wedge \neg C_9$	
$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{21} \wedge C_{37}$	

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.1 Autonomous vehicle — inv1: transition tick (4/4)

transition tick (7/7) 4 final passages exact cover (overlap permitted)

$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{37} \wedge C_9 \wedge C_{16}$ inv4

$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{37} \wedge C_9 \wedge \neg C_{16}$

$C_{13} \wedge \neg C_{14} \wedge C_{33} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{37} \wedge \neg C_9$

$\neg C_{13}$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.1 Autonomous vehicle — inv1: case-proposition key

C_1 $c\text{-a}(p1,s)$

C_5 $p1 = q$

C_9 $x(q,s) < x(q',s)$

C_{13} $c\text{-tick}(t1,s)$

C_{17} $loc(q',s) = accel$

C_{21} $loc(q',s) = brake$

C_{25}

$((t1 + v(q,a)) + ((1/2 * (t1 + t1)) + x(q',a)) + ((t1 + v(q',a)) + ((1/2 * (t1 + t1)) + x(q',a)))$

C_{29} $(x(q,s) + (t1 * v(q,s))) <$

0

C_{33} $loc(q,s) = brake$

C_{37}

$((t1 + v(q,a)) + ((1/2 * (t1 + t1)) + x(q',a)) + ((t1 + v(q',a)) + ((1/2 * (t1 + t1)) + x(q',a)))$

C_2 $c\text{-b}(p1,s)$

C_6 $ps(s) = (q' qs')$

C_{10} $c\text{-enter}(p1,s)$

C_{14} $loc(q,s) = start$

C_{18}

$0 < ((t1 + v(q',a)) + ((1/2 * (t1 + t1)) + x(q',a)))$

C_{22}

$0 < ((t1 + v(q',a)) + ((-1/2 * (t1 + t1)) + x(q',a)))$

C_{26}

$((t1 + v(q,a)) + ((1/2 * (t1 + t1)) + x(q',a)) < (x(q',a) + (t1 + v(q',a)))$

C_{30}

$(x(q,a) + (t1 + v(q,a))) < ((t1 + v(q',a)) + ((1/2 * (t1 + t1)) + x(q',a)))$

C_{34}

$((t1 + v(q,a)) + ((-1/2 * (t1 + t1)) + x(q,a))) < 0$

C_3 $loc(p1,s) = start$

C_7 $x(q',s) > 0$

C_{11} $ps(s) = nil$

C_{15} $loc(q',s) = start$

C_{19} $loc(q',s) = nothing$

C_{23} $loc(q,s) = accel$

C_{27}

$((t1 + v(q,a)) + ((1/2 * (t1 + t1)) + x(q',a)) + ((t1 + v(q',a)) + ((-1/2 * (t1 + t1)) + x(q',a)))$

C_{31}

$(x(q,a) + (t1 + v(q,a))) < (x(q',a) + (t1 + v(q',a)))$

C_{35}

$((t1 + v(q,a)) + ((-1/2 * (t1 + t1)) + x(q',a)) + ((t1 + v(q',a)) + ((1/2 * (t1 + t1)) + x(q',a)))$

C_4 $x(last(ps(s)),s) > 0$

C_8 $((q q') qs') \text{ sub } qs'$

C_{12} $c\text{-n}(p1,s)$

C_{16} $((q q') qs') \text{ sub } ps(s)$

C_{20} $0 < (x(q',s) + (t1 * v(q',s)))$

$v(q',s))$

C_{24}

$((t1 + v(q,a)) + ((1/2 * (t1 + t1)) + x(q,a))) < 0$

C_{28} $loc(q,s) = nothing$

C_{32}

$(x(q,a) + (t1 + v(q,a))) < ((t1 + v(q',a)) + ((-1/2 * (t1 + t1)) + x(q',a)))$

C_{36}

$((t1 + v(q,a)) + ((-1/2 * (t1 + t1)) + x(q,a))) < (x(q',a) + (t1 + v(q',a)))$

A.3.2 Autonomous vehicle — inv2: non-tick transitions (1/2)

transition enter 5 final passages exact cover (overlap permitted)

$C_4 \wedge C_5 \wedge C_2$ ☐
 $C_4 \wedge C_5 \wedge \neg C_2$ ☐
 $C_6 \wedge C_7 \wedge C_2$ ☐
 $C_6 \wedge C_7 \wedge \neg C_2$ ☐
 $\neg C_6 (= \neg(C_4 \wedge (C_5 \vee C_7)))$ ☐

transition b

4 final passages exact cover

$\neg C_3 \wedge C_2$ ☐
 $C_3 \wedge C_2$ ☐
 $C_3 \wedge \neg C_2$ ☐
 $\neg C_3 \wedge \neg C_2$ ☐

transition a

3 final passages exact cover

$C_1 \wedge C_2$ ☐
 $C_1 \wedge \neg C_2$ ☐
 $\neg C_1$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.2 Autonomous vehicle — inv2: non-tick transitions (2/2)

transition **n**

3 final passages exact cover

$C_8 \wedge C_2$ ☐

$C_8 \wedge \neg C_2$ ☐

$\neg C_8$ ☐

transition **init**

1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.2 Autonomous vehicle — inv2: transition tick (1/1)

transition tick (1/2) 10 final passages exact cover (overlap permitted)

$C_9 \wedge C_{10} \wedge C_{11}$	☐
$C_9 \wedge C_{10} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge C_{10} \wedge \neg C_{11} \wedge C_{13}$	☐
$C_9 \wedge C_{10} \wedge \neg C_{11} \wedge C_{14}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge C_{11}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge C_{13}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge C_{14}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge C_{11}$	☐

transition tick (2/2) 8 final passages exact cover (overlap permitted)

$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge \neg C_{11} \wedge C_{13}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge \neg C_{11} \wedge C_{14}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge C_{11}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge \neg C_{11} \wedge C_{13}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge \neg C_{11} \wedge C_{14}$	☐
$\neg C_9$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.2 Autonomous vehicle — inv2: case-proposition key

C_1 $c\text{-}a(p1,s)$

C_4 $loc(p1,s) = \text{start}$

C_7 $ps(s) = \text{nil}$

C_{10} $loc(q,s) = \text{start}$

C_{13} $loc(q',s) = \text{nothing}$

C_{16} $loc(q,s) = \text{nothing}$

C_2 $p1 = q$

C_5 $x(\text{last}(ps(s)),s) > 0$

C_8 $c\text{-}n(p1,s)$

C_{11} $loc(q',s) = \text{start}$

C_{14} $loc(q',s) = \text{brake}$

C_{17} $loc(q,s) = \text{brake}$

C_3 $c\text{-}b(p1,s)$

C_6 $c\text{-}enter(p1,s)$

C_9 $c\text{-}tick(t1,s)$

C_{12} $loc(q',s) = \text{accel}$

C_{15} $loc(q,s) = \text{accel}$

A.3.3 Autonomous vehicle — inv3: non-tick transitions (1/2)

transition **enter** 5 final passages exact cover (overlap permitted)

$C_4 \wedge C_5 \wedge C_2$ inv2, inv5

$C_4 \wedge C_5 \wedge \neg C_2$

$C_6 \wedge C_7 \wedge C_2$ inv2, inv5

$C_6 \wedge C_7 \wedge \neg C_2$

$\neg C_6 (= \neg(C_4 \wedge (C_5 \vee C_7)))$

transition **a**

3 final passages exact cover

$C_1 \wedge C_2$

$C_1 \wedge \neg C_2$

$\neg C_1$

transition **b**

3 final passages exact cover

$C_3 \wedge C_2$

$C_3 \wedge \neg C_2$

$\neg C_3$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.3 Autonomous vehicle — inv3: non-tick transitions (2/2)

transition **n**

3 final passages exact cover

$C_8 \wedge C_2$ ☐

$C_8 \wedge \neg C_2$ ☐

$\neg C_8$ ☐

transition **init**

1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.3 Autonomous vehicle — inv3: transition tick (1/1)

transition tick (1/1)

3 final passages exact cover

$C_9 \wedge C_{10}$ ☐

$C_9 \wedge \neg C_{10}$ ☐

$\neg C_9$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.3 Autonomous vehicle — inv3: case-proposition key

C_1 $c\text{-}a(p1,s)$

C_4 $loc(p1,s) = start$

C_7 $ps(s) = nil$

C_{10} $q' \text{ in } ps(s)$

C_2 $p1 = q$

C_5 $x(last(ps(s)),s) > 0$

C_8 $c\text{-}n(p1,s)$

C_3 $c\text{-}b(p1,s)$

C_6 $c\text{-}enter(p1,s)$

C_9 $c\text{-}tick(t1,s)$

A.3.4 Autonomous vehicle — inv4: non-tick transitions (1/2)

transition enter 10 final passages exact cover (overlap permitted)

$C_3 \wedge C_4 \wedge \neg C_5 \wedge C_6 \wedge C_7 \wedge C_8 \wedge C_9$ ☐

$C_3 \wedge C_4 \wedge \neg C_5 \wedge C_6 \wedge C_7 \wedge C_8 \wedge \neg C_9 \wedge C_{10}$ ☐

$C_3 \wedge C_4 \wedge \neg C_5 \wedge C_6 \wedge C_7 \wedge C_8 \wedge \neg C_9 \wedge \neg C_{10}$ ☐ inv2, inv5

$C_3 \wedge C_4 \wedge \neg C_5 \wedge C_6 \wedge C_7 \wedge \neg C_8$ ☐

$C_3 \wedge C_4 \wedge \neg C_5 \wedge C_6 \wedge \neg C_7$ ☐

$C_3 \wedge C_4 \wedge C_5 \wedge \neg C_6$ ☐

$C_3 \wedge C_4 \wedge C_5 \wedge C_6$ ☐

$C_3 \wedge C_4 \wedge \neg C_5 \wedge \neg C_6$ ☐

$C_{11} \wedge C_{12}$ ☐

$\neg C_{11} (= \neg(C_3 \wedge (C_4 \vee C_{12})))$ ☐

transition a

2 final passages exact cover

C_1 ☐

$\neg C_1$ ☐

transition b

2 final passages exact cover

C_2 ☐

$\neg C_2$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.4 Autonomous vehicle — inv4: non-tick transitions (2/2)

transition **n**

2 final passages exact cover



transition **init**

1 final passages split tree



Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.4 Autonomous vehicle — inv4: transition tick (1/3)

transition tick (1/6) 10 final passages exact cover (overlap permitted)

$C_{14} \wedge C_{15} \wedge C_{16} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge C_{15} \wedge C_{16} \wedge \neg C_{17}$	
$C_{14} \wedge C_{15} \wedge \neg C_{16} \wedge C_{18} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge C_{15} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{17}$	
$C_{14} \wedge C_{15} \wedge \neg C_{16} \wedge C_{19} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge C_{15} \wedge \neg C_{16} \wedge C_{19} \wedge \neg C_{17}$	
$C_{14} \wedge C_{15} \wedge \neg C_{16} \wedge C_{20} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge C_{15} \wedge \neg C_{16} \wedge C_{20} \wedge \neg C_{17}$	
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{16} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{16} \wedge \neg C_{17}$	

transition tick (2/6) 10 final passages exact cover (overlap permitted)

$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{16} \wedge C_{18} \wedge C_{22}$	
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{16} \wedge C_{18} \wedge C_{10} \wedge C_{17}$	
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{22} \wedge C_{10} \wedge \neg C_{17}$	
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{22} \wedge \neg C_{10}$	
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{19} \wedge C_{23}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{19} \wedge C_{10} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{19} \wedge \neg C_{23} \wedge C_{10} \wedge \neg C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{19} \wedge \neg C_{23} \wedge \neg C_{10}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{20} \wedge C_{24}$	
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{20} \wedge C_{10} \wedge C_{17}$	

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.4 Autonomous vehicle — inv4: transition tick (2/3)

transition tick (3/6) 10 final passages exact cover (overlap permitted)

$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{20} \wedge \neg C_{24} \wedge C_{10} \wedge \neg C_{17}$	☐
$C_{14} \wedge \neg C_{15} \wedge C_{21} \wedge C_{20} \wedge \neg C_{24} \wedge \neg C_{10}$	☐
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge C_{16} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge C_{16} \wedge \neg C_{17}$	☐
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{18} \wedge C_{26}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{18} \wedge C_{27} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{26} \wedge C_{27} \wedge \neg C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{26} \wedge \neg C_{27}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{19} \wedge C_{28}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{19} \wedge C_{27}$	inv3, inv6

transition tick (4/6) 10 final passages exact cover (overlap permitted)

$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{19} \wedge \neg C_{28} \wedge C_{27} \wedge \neg C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{19} \wedge \neg C_{28} \wedge \neg C_{27}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{19} \wedge \neg C_{27}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{20} \wedge C_{29}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{20} \wedge C_{10} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{20} \wedge \neg C_{29} \wedge C_{10} \wedge \neg C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{25} \wedge \neg C_{16} \wedge C_{20} \wedge \neg C_{29} \wedge \neg C_{10}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge C_{16} \wedge \neg C_{17}$	☐
$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge C_{16} \wedge C_{17}$	inv3, inv6
$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{18} \wedge C_{31}$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.4 Autonomous vehicle — inv4: transition tick (3/3)

transition tick (5/6) 10 final passages exact cover (overlap permitted)

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{18} \wedge C_{10} \wedge C_{17} \quad \boxed{\text{inv6}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{31} \wedge C_{10} \wedge \neg C_{17} \quad \boxed{\phantom{\text{inv6}}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{18} \wedge \neg C_{31} \wedge \neg C_{10} \quad \boxed{\phantom{\text{inv6}}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{19} \wedge C_{32} \quad \boxed{\phantom{\text{inv6}}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{19} \wedge C_{10} \wedge C_{17} \quad \boxed{\text{inv6}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{19} \wedge \neg C_{32} \wedge C_{10} \wedge \neg C_{17} \quad \boxed{\phantom{\text{inv6}}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{19} \wedge \neg C_{32} \wedge \neg C_{10} \quad \boxed{\phantom{\text{inv6}}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{20} \wedge C_{33} \quad \boxed{\phantom{\text{inv6}}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{20} \wedge C_{10} \wedge C_{17} \quad \boxed{\text{inv3, inv6}}$$

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{20} \wedge \neg C_{33} \wedge C_{10} \wedge \neg C_{17} \quad \boxed{\phantom{\text{inv6}}}$$

transition tick (6/6) 2 final passages exact cover (overlap permitted)

$$C_{14} \wedge \neg C_{15} \wedge C_{30} \wedge \neg C_{16} \wedge C_{20} \wedge \neg C_{33} \wedge \neg C_{10} \quad \boxed{\phantom{\text{inv6}}}$$

$$\neg C_{14} \quad \boxed{\phantom{\text{inv6}}}$$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.5 Autonomous vehicle — inv5: non-tick transitions (1/2)

transition enter 5 final passages exact cover (overlap permitted)

$C_4 \wedge C_5 \wedge C_2$ ☐
 $C_4 \wedge C_5 \wedge \neg C_2$ ☐
 $C_6 \wedge C_7 \wedge C_2$ ☐
 $C_6 \wedge C_7 \wedge \neg C_2$ ☐
 $\neg C_6 (= \neg(C_4 \wedge (C_5 \vee C_7)))$ ☐

transition b

4 final passages exact cover

$\neg C_3 \wedge C_2$ ☐
 $C_3 \wedge C_2$ ☐
 $C_3 \wedge \neg C_2$ ☐
 $\neg C_3 \wedge \neg C_2$ ☐

transition a

3 final passages exact cover

$C_1 \wedge C_2$ ☐
 $C_1 \wedge \neg C_2$ ☐
 $\neg C_1$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.5 Autonomous vehicle — inv5: non-tick transitions (2/2)

transition **n**

3 final passages exact cover

$C_8 \wedge C_2$ ☐

$C_8 \wedge \neg C_2$ ☐

$\neg C_8$ ☐

transition **init**

1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.5 Autonomous vehicle — inv5: transition tick (1/1)

transition tick (1/2) 10 final passages exact cover (overlap permitted)

$C_9 \wedge C_{10} \wedge C_{11}$	☐
$C_9 \wedge C_{10} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge C_{10} \wedge \neg C_{11} \wedge C_{13}$	☐
$C_9 \wedge C_{10} \wedge \neg C_{11} \wedge C_{14}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge C_{11}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge C_{13}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{15} \wedge C_{14}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge C_{11}$	☐

transition tick (2/2) 8 final passages exact cover (overlap permitted)

$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge \neg C_{11} \wedge C_{13}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{16} \wedge \neg C_{11} \wedge C_{14}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge C_{11}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge \neg C_{11} \wedge C_{12}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge \neg C_{11} \wedge C_{13}$	☐
$C_9 \wedge \neg C_{10} \wedge C_{17} \wedge \neg C_{11} \wedge C_{14}$	☐
$\neg C_9$	☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.5 Autonomous vehicle — inv5: case-proposition key

C_1 $c\text{-}a(p1,s)$

C_4 $loc(p1,s) = \text{start}$

C_7 $ps(s) = \text{nil}$

C_{10} $loc(q,s) = \text{start}$

C_{13} $loc(q',s) = \text{nothing}$

C_{16} $loc(q,s) = \text{nothing}$

C_2 $p1 = q$

C_5 $x(\text{last}(ps(s)),s) > 0$

C_8 $c\text{-}n(p1,s)$

C_{11} $loc(q',s) = \text{start}$

C_{14} $loc(q',s) = \text{brake}$

C_{17} $loc(q,s) = \text{brake}$

C_3 $c\text{-}b(p1,s)$

C_6 $c\text{-}enter(p1,s)$

C_9 $c\text{-}tick(t1,s)$

C_{12} $loc(q',s) = \text{accel}$

C_{15} $loc(q,s) = \text{accel}$

A.3.6 Autonomous vehicle — inv6: non-tick transitions (1/2)

transition **enter** 5 final passages exact cover (overlap permitted)

$C_4 \wedge C_5 \wedge C_2$ inv2, inv5

$C_4 \wedge C_5 \wedge \neg C_2$

$C_6 \wedge C_7 \wedge C_2$ inv2, inv5

$C_6 \wedge C_7 \wedge \neg C_2$

$\neg C_6 (= \neg(C_4 \wedge (C_5 \vee C_7)))$

transition **a**

3 final passages exact cover

$C_1 \wedge C_2$

$C_1 \wedge \neg C_2$

$\neg C_1$

transition **b**

3 final passages exact cover

$C_3 \wedge C_2$

$C_3 \wedge \neg C_2$

$\neg C_3$

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.6 Autonomous vehicle — inv6: non-tick transitions (2/2)

transition **n**

3 final passages exact cover

$C_8 \wedge C_2$ ☐

$C_8 \wedge \neg C_2$ ☐

$\neg C_8$ ☐

transition **init**

1 final passages split tree

$\top$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.6 Autonomous vehicle — inv6: transition tick (1/1)

transition tick (1/1)

4 final passages exact cover

$C_9 \wedge C_{10} \wedge C_{11}$ ☐

$C_9 \wedge C_{10} \wedge \neg C_{11}$ ☐

$C_9 \wedge \neg C_{10}$ ☐

$\neg C_9$ ☐

Each row is one final root-to-leaf case. The right-hand terminal box is empty unless that passage uses an invariant lemma.

A.3.6 Autonomous vehicle — inv6: case-proposition key

C_1 $c\text{-}a(p1,s)$

C_4 $loc(p1,s) = \text{start}$

C_7 $ps(s) = \text{nil}$

C_{10} $q' \text{ in } ps(s)$

C_2 $p1 = q$

C_5 $x(\text{last}(ps(s)),s) > 0$

C_8 $c\text{-}n(p1,s)$

C_{11} $loc(q',s) = \text{start}$

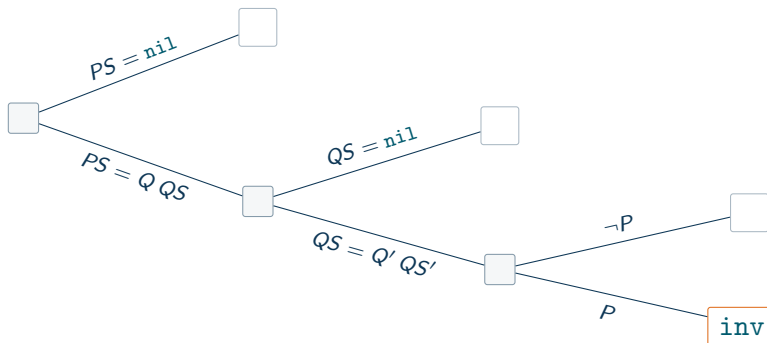
C_3 $c\text{-}b(p1,s)$

C_6 $c\text{-}enter(p1,s)$

C_9 $c\text{-}tick(t1,s)$

A.4 Vehicle PSet safety — structural induction

The structural-safety proof follows the base/constructor decomposition of **PSet** terms.



inv1 P is the nontrivial adjacent-pair premise. Only that final constructor passage uses **inv1**.

A.5 Invariant dependencies in the accompanying Proof Scores

An arrow $\text{invA} \longrightarrow \text{invB}$ means that at least one proof passage for invA actually uses invB as a lemma.

ABCf

$\text{inv1} \longrightarrow \text{inv2}$
 $\text{inv2} \longrightarrow \text{inv1}$

Fischer

$\text{inv1} \longrightarrow \text{inv2}$
 $\text{inv2} \longrightarrow \text{inv3}$
 $\text{inv2} \longrightarrow \text{inv5}$
 $\text{inv3} \longrightarrow \text{inv4}$

Autonomous vehicle

$\text{inv1} \longrightarrow \text{inv2}$
 $\text{inv1} \longrightarrow \text{inv3}$
 $\text{inv1} \longrightarrow \text{inv4}$
 $\text{inv1} \longrightarrow \text{inv5}$
 $\text{inv1} \longrightarrow \text{inv6}$
 $\text{inv3} \longrightarrow \text{inv2}$
 $\text{inv3} \longrightarrow \text{inv5}$
 $\text{inv4} \longrightarrow \text{inv2}$
 $\text{inv4} \longrightarrow \text{inv3}$
 $\text{inv4} \longrightarrow \text{inv5}$
 $\text{inv4} \longrightarrow \text{inv6}$
 $\text{inv6} \longrightarrow \text{inv2}$
 $\text{inv6} \longrightarrow \text{inv5}$

The graph is derived from the same leaf records used in the maps.